

Density Decomposition on Hypergraphs

ABSTRACT

Decomposing hypergraphs is a key task in hypergraph analysis with broad applications in community detection, pattern discovery, and task scheduling. However, existing models like k -core and neighbor- k -core focus on the minimum degree of vertices rather than density-based decomposition. This can lead to sparse layers or uneven density distribution between layers. To address these issues, we propose a novel (k, δ) -dense subhypergraph model for decomposing hypergraphs based on integer density values. Here, k represents the subgraph's integer density, and δ sets the upper limit for each hyperedge's contribution to the density. Identifying such subhypergraphs requires computing an egalitarian orientation, with intuitive complexity up to $O(2^{m\delta})$. To enable efficient computation, we propose a fair-stable-based algorithm that reduces the time complexity of finding a single (k, δ) -dense subhypergraph from $O(m^2\delta^2)$ to $O(nm\delta)$. Furthermore, we design a divide-and-conquer decomposition framework that improves the overall complexity of full density decomposition from $O(nm\delta \cdot d_{\max}^E \cdot k_{\max})$ to $O(nm\delta \cdot d_{\max}^E \cdot \log k_{\max})$. Experiments across nine real-world hypergraph datasets demonstrate that our method consistently outperforms previous core-based approaches in terms of decomposition continuity and non-redundancy, while maintaining strong computational efficiency. Case studies further demonstrate the practical value of our framework, revealing cohesive and interpretable community structures that core-based methods often overlook.

CCS CONCEPTS

• Information systems → Graph-based database models.

KEYWORDS

hypergraph, density, density decomposition

ACM Reference Format:

. 2018. Density Decomposition on Hypergraphs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Hypergraphs naturally model multi-way relationships in real-world systems, such as multi-party transactions in financial networks, group interactions in social platforms, and collaborative activities in recommendation systems. These higher-order structures offer richer expressive power than simple graphs. A central task in hypergraph analysis is to decompose dense subhypergraphs—regions

with strong internal connectivity—which often correspond to tightly-knit communities or functional units. Such decompositions are crucial for discovering cohesive groups, summarizing multi-way behaviors, and enabling interpretable analysis in domains where relations naturally involve more than two participants [2, 12, 27, 30, 32, 33].

Among the recent researches, k -core is the most typical kind of decomposition, which requires all vertices' degrees to be no less than k [27]. However, k -core decomposition may fail to achieve a good subhypergraph decomposition, as the number of layers in this decomposition is often related to k . Given that k is usually not large in real applications, the number of layers in k -core decomposition is often limited. As shown in Figure 1(b), the entire hypergraph is divided into two layers, where all vertices in layer C_1 have a degree of at least 1, and all vertices in layer C_2 have a degree of at least 2. Therefore, how can we find a decomposition that better targets the size of density? The effect of our proposed method can be seen in Figure 1(a), where the hypergraph is eventually decomposed into four layers. The subhypergraph in $D_{4,4}$ has a density of 4; the subhypergraph in $D_{3,4} \setminus D_{4,4}$ has a density of 3; the subhypergraph in $D_{1,4} \setminus D_{2,4}$ has a density of 1. More excitingly, our proposed method can decompose this hypergraph according to the integer value of density. To our best knowledge, this is the first work on hypergraph decomposition based on density. (There is a recent important work, the nbr- k -core model [2, 43], which also performs hypergraph decomposition, but it mainly uses the degree of neighbors for decomposition, not density.)

Motivation. Despite the success of core-based or nbr-core-based decompositions, these models inherently depend on local degree thresholds rather than true structural density. As a result, they tend to produce coarse layers with uneven density distribution, making it difficult to reveal fine-grained, hierarchically nested communities that are crucial in many real-world applications. In particular, many hypergraphs—such as *financial transaction networks*, *legislative co-sponsorship graphs*, and *biological interaction systems*—exhibit overlapping and multi-party relationships [12], where identifying cohesive and hierarchically organized groups is vital for downstream tasks [30, 31, 39] (e.g., fraud detection, influence analysis, and function discovery). However, degree-based decompositions fail to distinguish such patterns, often merging dense and sparse regions together. Addressing this limitation requires a decomposition model that directly targets *density itself* rather than vertex degree, and does so in a computationally efficient and interpretable manner. The case studies presented later in this paper (Section 6.4) further demonstrate how density-driven decomposition reveals meaningful structures—such as cross-party alliances in legislative networks and compact suspicious-account clusters in anti-money-laundering scenarios—that core-based methods completely overlook.

To decompose hypergraphs by density, we propose the (k, δ) -dense subhypergraph model. In this model, each vertex must accumulate at least k units of indegree (under a constrained orientation) or be reachable from such a vertex via a contribution-preserving hyperpath. This formulation unifies structural and directional density constraints, offering a more nuanced characterization of dense

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

regions. We further show that (k, δ) -dense subhypergraphs form a nested and hierarchical decomposition of the hypergraph, which we refer to as the (k, δ) -density decomposition. This decomposition supports fine-grained structural analysis, enabling scalable, multi-resolution exploration of hypergraphs.

To bridge the gap between degree-based and truly density-oriented decompositions, we propose a principled framework that characterizes hypergraph cohesiveness through integer-valued density levels, leading to a new notion of hierarchical and interpretable structure discovery. Our contributions are summarized as follows:

- We propose the (k, δ) -dense subhypergraph model, which unifies vertex and structural density constraints to enable fine-grained hierarchical decomposition of hypergraphs. Interestingly, both k and δ hold significant practical implications: k represents the integer density level at each layer, while δ denotes the upper limit of each hyperedge's contribution to the density. Compared with the state-of-the-art *nbr-k-core* frameworks [2, 43], our model introduces a fundamentally different decomposition principle. The *nbr-k-core* model [2] decomposes hypergraphs based on the minimum degrees of neighbors, while its large-scale variant [43] improves computational scalability but retains the same degree-based criterion. Such formulations cannot reflect the true density of multi-way relationships and often produce coarse or uneven layers. In contrast, our (k, δ) -dense model directly decomposes a hypergraph by integer-valued density levels, yielding smoother hierarchical transitions, stronger interpretability, and natural extensibility to bipartite graphs as a special case (Section 2.7).
- We propose a hyperpath-based approach for mining (k, δ) -dense subhypergraphs, and further develop a flow-based method that improves the complexity from $O(m^2\delta^2)$ to $O(m^{1.5}\bar{d}_e^{1.5})$, where n and m denote the numbers of vertices and hyperedges and $\bar{d}_e$ is the average hyperedge size. Building further, we introduce a fair-stable method with an improved time complexity of $O(nm\delta)$. Building on this, we design two efficient decomposition algorithms: one based on a multi-layer peeling framework and another using divide-and-conquer, which reduce the overall decomposition cost from $O(nm\delta \cdot d_{max}^E \cdot k_{max})$ to $O(nm\delta \cdot d_{max}^E \cdot \log k_{max})$, where d_{max}^E is the maximum hyperedge size and k_{max} is the maximum density level explored in the decomposition.
- We conduct extensive experiments on nine real-world hypergraph datasets to evaluate the effectiveness and efficiency of our proposed methods. The results demonstrate that our approach consistently outperforms the state-of-the-art *nbr-k-core* decomposition algorithms [2, 43], our proposed method achieves a 20-fold increase in the number of decomposition layers and a 10× improvement in computational efficiency.
- We release an anonymous and reproducible implementation at: <https://anonymous.4open.science/r/IDH-8ED9>.

2 PROBLEM DEFINITION

An undirected and unweighted hypergraph is defined as $\mathcal{H} = (V, E)$, where V is the set of vertices and E is the set of hyperedges, with each hyperedge $e \in E$ being a subset of V (i.e., $e \subseteq V$ and $|e| \geq 1$). Let $n = |V|$ and $m = |E|$ denote the numbers of vertices

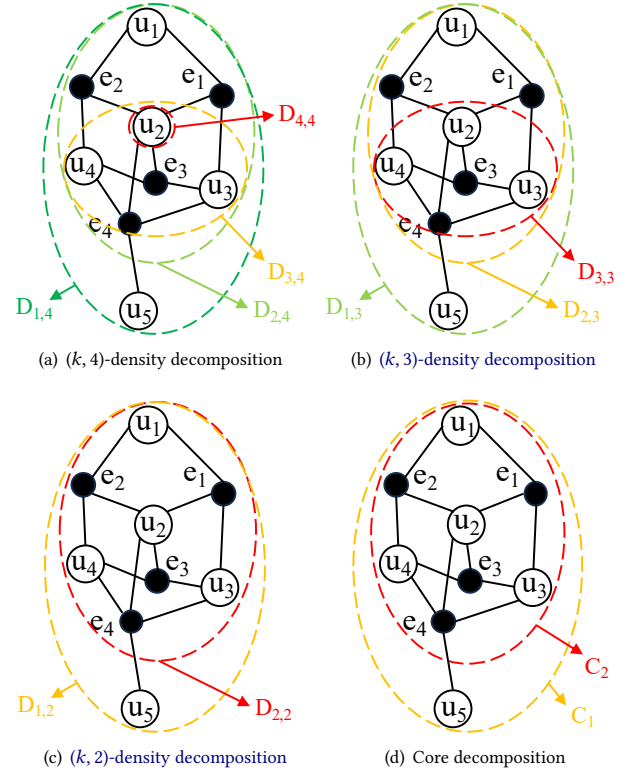


Figure 1: Density decomposition and core decomposition. Each e_i denotes a hyperedge, each u_j denotes a vertex. An “edge” between e_i and u_j indicates that vertex u_j is contained in hyperedge e_i . (Note: this is a schematic representation.) This figure illustrates how varying δ (2, 3, 4) produces different levels of decomposition granularity: smaller δ yields coarser partitions, while larger δ highlights finer-grained dense layers.

and hyperedges, respectively. We denote by d_{max}^E and d_{min}^E the maximum and minimum hyperedge sizes in $\mathcal{H}$, and by $\bar{d}_e = \frac{1}{m} \sum_{e \in E} |e|$ the average hyperedge size. A parameter $\delta \in [1, d_{max}^E]$ is introduced to cap the contribution of each hyperedge in the density formulation. The degree of a vertex $u \in V$, denoted d_u , is the number of hyperedges that contain u , and d_{max}^V represents the maximum vertex degree. For a subset $S \subseteq V$, we use $E[S]$ to denote the set of hyperedges entirely contained within S . These definitions establish the notation for dense structures in hypergraphs.

2.1 Directed Hypergraphs and Orientations

To capture directional relationships, we introduce orientations of hyperedges. Assigning a direction to each hyperedge of a hypergraph $\mathcal{H} = (V, E)$, transforms it into a directed hypergraph termed as an *orientation* of $\mathcal{H}$, denoted by $\vec{\mathcal{H}} = (V, \vec{E})$, where $\vec{E}$ comprises the directed hyperedges. For example, the directed hypergraphs depicted in Figure 2(b)–(c) are orientations of the undirected hypergraph shown in Figure 2(a). We use $\vec{e} = (T, H)$ ($T = \{v_{x_1}, v_{x_2}, \dots, v_{x_i}\}$, $H = \{v_{x_{i+1}}, v_{x_{i+2}}, \dots, v_{x_n}\}$) to denote a directed hyperedge. T is a set represents the set of source vertices of this hyperedge, while

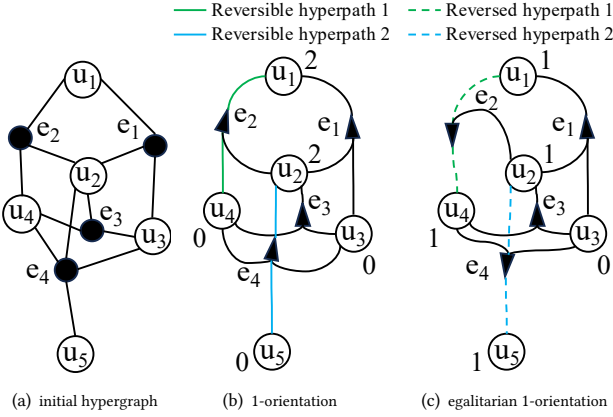


Figure 2: Arbitrary and egalitarian 1-orientation.

H represents the set of target vertices. For example, directed hyperedge $\vec{e}_4 = (\{u_3, u_4, u_5\}, \{u_2\})$ in Figure 2(b), vertices u_3, u_4, u_5 are sources vertices of $\vec{e}_4$, vertex u_2 is target vertex. In the orientation $\vec{\mathcal{H}}$, the indegree of a vertex $u \in V$ is denoted by $\vec{d}_u(\vec{\mathcal{H}}) = |\{u | u \in H, (T, H) \in \vec{E}\}|$ or, more succinctly, $\vec{d}_u$.

DEFINITION 1 (δ -ORIENTATION). Given a hypergraph $\mathcal{H} = (V, E)$ and its orientation $\vec{\mathcal{H}} = (V, \vec{E})$, if for each directed hyperedge $\vec{e} = (T, H) \in \vec{E}$, we have $|H| = \min\{\delta, |e|\}$, where $\delta \in [1, d_{max}^E]$, then $\vec{\mathcal{H}}$ is a δ -orientation of $\mathcal{H}$.

2.2 Hyperpaths and Egalitarian Orientation

Before introducing the density decomposition, we define hyperpath in a directed hypergraph.

DEFINITION 2 (HYPERPATH AND REVERSIBLE HYPERPATH). In a directed hypergraph $\vec{\mathcal{H}} = (V, \vec{E})$, a hyperpath from vertex s to t is a sequence: $s = u_0, \vec{e}_0, u_1, \dots, \vec{e}_{l-1}, u_l = t$, where each $\vec{e}_i = (T_i, H_i)$ satisfies $u_i \in T_i$ and $u_{i+1} \in H_i$. The hyperpath is reversible if $\vec{d}_t - \vec{d}_s \geq 2$.

If a hyperpath $s \rightsquigarrow t$ exists, then we say that s can reach t . When a hyperpath is reversed, all hyperedges and vertices in the hyperpath undergo a reversal. Intuitively, an egalitarian δ -orientation distributes the indegree of all vertices in the most equitable manner, i.e., minimizing the indegree difference between vertices as much as possible. Note that if reverse a reversible hyperpath $s \rightsquigarrow t$, $\vec{d}_s$ increases by 1, $\vec{d}_t$ decreases by 1, and the indegree of other vertices does not change, making the indegree difference between s and t reduced by 2. When no reversible hyperpath exists, the indegree difference can not be reduced anymore.

DEFINITION 3 (EGALITARIAN δ -ORIENTATION). A δ -orientation $\vec{\mathcal{H}}$ is said to be egalitarian δ -orientation hypergraph if there exists no reversible hyperpath in $\vec{\mathcal{H}}$.

EXAMPLE 1. Figure 2(b) illustrates an arbitrary 1-orientation hypergraph in which there exists hyperpaths $u_4 \rightsquigarrow u_1$ ($u_4, \vec{e}_2, u_1$) and $u_5 \rightsquigarrow u_2$ ($u_5, \vec{e}_4, u_2$). The indegree difference between u_4 and u_1 is

equal to 2, the same to u_5 and u_2 . Therefore, the hyperpath $u_4 \rightsquigarrow u_1$ and hyperpath $u_5 \rightsquigarrow u_2$ are reversible hyperpaths. In Figure 2(c), we reverse the two reversible hyperpaths by inverting the direction of all hyperedges along the hyperpaths, resulting in the hyperpaths $u_1 \rightsquigarrow u_4$ ($u_1, \vec{e}_2, u_4$) and $u_2 \rightsquigarrow u_5$ ($u_2, \vec{e}_4, u_5$). This reversal reduces the indegree difference between u_4 and u_1 by 2, the same to u_5 and u_2 . Notably, after these operations, no reversible hyperpaths remain in the hypergraph, indicating that the hypergraph is an egalitarian 1-orientation hypergraph.

2.3 (k, δ) -Dense Subhypergraph

We now define our density-based model for subhypergraph:

DEFINITION 4 ((k, δ) -DENSE SUBHYPERGRAPH $(D_{k, \delta})$). Given an undirected and unweighted hypergraph $\mathcal{H} = (V, E)$, two non-negative integers k and δ , let $\vec{\mathcal{H}}$ be an arbitrary egalitarian δ -orientation of $\mathcal{H}$, and let $S = \{u \in V | \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$, the (k, δ) -dense subhypergraph $D_{k, \delta}$ is the subhypergraph induced by the vertices set $S \cup \{v | v \text{ can reach a vertex in } S\}$.

EXAMPLE 2. Consider the egalitarian 1-orientation in Figure 2(c). Let $k = 1$, then $S = \{u_1, u_2, u_4, u_5\}$. The vertex u_3 can reach u_2 via the hyperpath $(u_3, \vec{e}_3, u_2)$, so the $(1, 1)$ -dense subhypergraph includes $\{u_1, u_2, u_3, u_4, u_5\}$.

By Definition 4, we can easily obtain the following results.

LEMMA 1. Given an egalitarian δ -orientation $\vec{\mathcal{H}}$ and the corresponding (k, δ) -dense subhypergraph $D_{k, \delta}$:

- 1) All hyperedges crossing from $D_{k, \delta}$ to $V \setminus D_{k, \delta}$ are oriented outward.
- 2) For any $u \in D_{k, \delta}$, we have $\vec{d}_u \geq k - 1$.
- 3) For any $u \notin D_{k, \delta}$, we have $\vec{d}_u \leq k - 1$.

The following theorem confirms that $D_{k, \delta}$ is internally cohesive and externally separated.

THEOREM 1. Let $D_{k, \delta}$ be a (k, δ) -dense subhypergraph. Then:

- For any non-empty $X \subseteq D_{k, \delta}$, $\sum_{x \in X} \vec{d}_x > (k - 1)|X|$.
- For any $Y \subseteq V \setminus D_{k, \delta}$, $\sum_{y \in Y} \vec{d}_y \leq (k - 1)|Y|$.

PROOF. The first claim follows from Lemma 1 and Definition 4: all hyperedges contributing to the indegree of X lie within $D_{k, \delta}$. The inequality must be strict to ensure the reachability condition is satisfied. The second claim follows from the upper bound on indegrees for vertices outside $D_{k, \delta}$. $\square$

Remark on δ . The parameter δ serves as a resolution control that governs the granularity of the identified structures. By limiting the maximum contribution of each hyperedge to the density, it determines how localized or global the resulting dense regions appear. A larger δ accentuates fine-grained, cohesive clusters, while a smaller δ aggregates them into broader, coarser structures. This effect can be observed in Figure 1, where $\delta=2,3,4$ produce progressively different decomposition granularities. This tunable δ parameter allows analysts to smoothly adjust between coarse-grained and fine-grained views of the same hypergraph, which is essential for multi-resolution analysis in real-world systems.

2.4 Properties of the Decomposition

We now analyze the theoretical properties of the (k, δ) -dense subhypergraph model and its induced decomposition.

Uniqueness of Decomposition. The definition of $D_{k,\delta}$ relies only on the existence of an egalitarian δ -orientation. Therefore, regardless of which egalitarian orientation is chosen, the resulting $D_{k,\delta}$ remains the same.

THEOREM 2. *Given a hypergraph $\mathcal{H}$ and parameters k, δ , the $D_{k,\delta}$ is unique.*

PROOF. Suppose two different decompositions $D_{a,\delta}$ and $D_{b,\delta}$ exist, with non-empty difference $D = D_{a,\delta} \setminus D_{b,\delta}$. According to Theorem 1, we obtain contradictory bounds on the aggregate indegree of vertices in D . Thus, $D_{k,\delta}$ must be unique. $\square$

Hierarchy of (k, δ) -Dense Subhypergraphs

THEOREM 3. *For any $k^+ \geq k$, we have $D_{k^+,\delta} \subseteq D_{k,\delta}$.*

PROOF. Assume $D = D_{k^+,\delta} \setminus D_{k,\delta} \neq \emptyset$. Using Theorem 1, we derive a contradiction by comparing lower and upper bounds on the indegree sum over D . $\square$

Density Metrics and Interpretability. We next analyze the (k, δ) -dense subhypergraph using degree-based density.

DEFINITION 5 (INDEGREE-BASED DENSITY). *Given a directed hypergraph $\vec{\mathcal{H}} = (V, \vec{E})$ and a vertex set X , its indegree-based density is defined as $\rho_d(X) = \sum_{x \in X} \vec{d}_x / |X|$.*

LEMMA 2. *For any $X \subseteq D_{k,\delta}$ and $Y \subseteq V \setminus D_{k,\delta}$, we have $\rho_d(X) > k - 1 \geq \rho_d(Y)$.*

PROOF. Directly follows from Theorem 1. $\square$

DEFINITION 6 (LAYER DENSITY). *For any non-negative integer k , we define the layer density as $\rho_d(L_{k,\delta}) = \rho_d(D_{k+1,\delta}, D_{k,\delta}) = \sum_{x \in D_{k,\delta} \setminus D_{k+1,\delta}} \vec{d}_x / |D_{k,\delta} \setminus D_{k+1,\delta}|$.*

LEMMA 3. *For any non-negative integer k , we have $(k - 1) < \rho_d(D_{k+1,\delta}, D_{k,\delta}) \leq k$. Thus, $\lceil \rho_d(L_{k,\delta}) \rceil = \lceil \rho_d(D_{k+1,\delta}, D_{k,\delta}) \rceil = k$, indicating that layer $L_{k,\delta}$ corresponds to density level k .*

PROOF. According to Theorem 1, for any vertex $x \in D_{k,\delta} \setminus D_{k+1,\delta}$, we have $\rho_d(x) \geq k - 1$. Furthermore, the total indegree over any non-empty subset of $D_{k,\delta}$ strictly exceeds $(k - 1) \cdot |X|$, ensuring that $\rho_d(D_{k+1,\delta}, D_{k,\delta}) > k - 1$. On the other hand, by the definition of $D_{k+1,\delta}$, all vertices not included in it must have indegree strictly less than $k + 1$, hence every $x \in D_{k,\delta} \setminus D_{k+1,\delta}$ satisfies $\rho_d(x) \leq k$, leading to $\rho_d(D_{k+1,\delta}, D_{k,\delta}) \leq k$. Therefore, the layer density lies in the interval $(k - 1, k]$, and it follows that $\lceil \rho_d(L_{k,\delta}) \rceil = k$. $\square$

DEFINITION 7 (INTEGRAL DENSE NUMBER (IDN)). *For $u \in D_{k,\delta} \setminus D_{k+1,\delta}$, define its IDN as $\tilde{r}_u^\delta = k$.*

Our proposed method can decompose this hypergraph according to the integer value of density.

In conclusion, the above three properties lead to the following implications:

(i) Due to the uniqueness property of the (k, δ) -dense subhypergraph, one can obtain a valid (k, δ) -dense subhypergraph by computing any egalitarian orientation, which enables efficient mining of such subhypergraphs;

(ii) Based on Lemma 3, the integer value of the density at each level of the (k, δ) -dense subhypergraph corresponds to the parameter k ;

(iii) Owing to the inheritance property of the (k, δ) -dense subhypergraph, a divide-and-conquer framework can be employed for hierarchical decomposition.

2.5 Density and conductance guarantee

Since the hypergraph to be decomposed is originally undirected, we define the degree-based density for undirected hypergraphs..

DEFINITION 8 (DEGREE-DENSITY). *Given a hypergraph $\mathcal{H}$ and a subhypergraph X , the density of X is defined as $\rho(X) = \frac{\sum_{x \in X} d_x(X)}{|X|}$.*

The subhypergraph $X \subseteq V$ that maximizes the density $\rho(X)$ is recognized as the *densest subhypergraph* of $\mathcal{H}$.

LEMMA 4. *For any $D_{k,\delta} \neq \emptyset$, we have $\rho(D_{k,\delta}) \geq \rho_d(D_{k,\delta}) > k - 1$.*

PROOF. Under an egalitarian δ -orientation, each internal hyperedge contributes at most one unit of indegree to a single vertex, while in $\rho(X)$ it contributes to all endpoints. Furthermore, cross hyperedges are oriented outward and do not increase indegree inside. Hence $\sum_{x \in X} \vec{d}_x \leq \sum_{x \in X} d_x(X)$. $\square$

DEFINITION 9 (INTERNALIZATION COEFFICIENT). *For a subhypergraph X , define $\theta(X) = \frac{\sum_{e \in E(X)} |H(e)|}{\sum_{x \in X} \vec{d}_x} \in [0, 1]$, where $\theta(X) = 0$ if $\sum_{x \in X} \vec{d}_x = 0$. Let $f_k = |S_k| / |D_{k,\delta}|$ denote the fraction within $D_{k,\delta}$.*

LEMMA 5. *For any k, δ with $D_{k,\delta} \neq \emptyset$, $\rho_d(D_{k,\delta}) \geq (k - 1) + f_k$.*

PROOF. By Definition 4, each vertex $s \in S_k$ has indegree at least k , and each vertex $x \in D_{k,\delta} \setminus S_k$ has indegree at least $k - 1$ but can reach some s . Thus $\rho_d(D_{k,\delta}) \geq f_k \cdot k + (1 - f_k)(k - 1) = (k - 1) + f_k$. $\square$

LEMMA 6. *For any $X \subseteq V$, $\rho(X) \geq \theta(X) \rho_d(X)$.*

PROOF. We have $\sum_{x \in X} d_x(X) = \sum_{e \in E(X)} |e| \geq \sum_{e \in E(X)} |H(e)| = \theta(X) \sum_{x \in X} \vec{d}_x$. Divide by $|X|$. $\square$

THEOREM 4 (DENSITY GUARANTEE). *For any k, δ , we have $\rho(D_{k,\delta}) \geq \theta(D_{k,\delta})((k - 1) + f_k)$.*

PROOF. Vertices in S_k satisfy $\vec{d} \geq k$, while vertices in $D_{k,\delta} \setminus S_k$ have $\vec{d} \geq k - 1$ and can reach some $s \in S_k$. Hence $\rho_d(D_{k,\delta}) \geq (k - 1) + f_k$. Moreover, $\rho(X) \geq \theta(X) \rho_d(X)$ since $\sum_{x \in X} d_x(X) = \sum_{e \in E(X)} |e| \geq \sum_{e \in E(X)} |H(e)| = \theta(X) \sum_{x \in X} \vec{d}_x$. Combining the two inequalities gives the bound. (Equivalently, in the edge/vertex view: $\frac{|E(D_{k,\delta})|}{|D_{k,\delta}|} = \frac{|E(D_{k,\delta})| \cdot \bar{r}(D_{k,\delta})}{|D_{k,\delta}| \cdot \bar{r}(D_{k,\delta})} = \frac{\rho(D_{k,\delta})}{\bar{r}(D_{k,\delta})} \geq \frac{\theta((k - 1) + f_k)}{\bar{r}(D_{k,\delta})}$.) $\square$

THEOREM 5 (CONDUCTANCE). *For any $X \subseteq V$, conductance $\phi(X) \leq 1 - \frac{\rho(X)}{\bar{d}(X)}$, where $\bar{d}(X) = \text{vol}(X) / |X|$. In particular, $\phi(D_{k,\delta}) \leq 1 - \frac{\theta(D_{k,\delta})((k - 1) + f_k)}{\bar{d}(D_{k,\delta})}$.*

PROOF. Each incident hyperedge of X is either internal or crossing. Thus, $\partial(X) \leq \text{vol}(X) - \rho(X)|X| \Rightarrow \phi(X) \leq 1 - \frac{\rho(X)}{d(X)}$. Substituting $X = D_{k,\delta}$ and applying the density guarantee yields the desired bound. $\square$

2.6 Extension to Bipartite Graphs

Having established the theoretical properties of (k, δ) -dense subhypergraphs, we next demonstrate that the proposed model generalizes seamlessly to bipartite graphs. A bipartite graph $G = (U, V, E)$ can be modeled as a hypergraph by taking the left partition U as a collection of hyperedges and the right partition V as the vertex set. Specifically, each $u \in U$ corresponds to a hyperedge e_u containing all its neighbors in V . Thus, the degree of a vertex $v \in V$ in the bipartite graph coincides with its degree in the derived hypergraph. Consequently, our (k, δ) -density decomposition applies directly:

- **Orientation:** for each hyperedge e_u , at most δ incident edges contribute indegree to its adjacent vertices in V .
- **Reversible hyperpaths and egalitarian orientation:** are defined and balanced exclusively among vertices in V .
- **Decomposition:** $D_{k,\delta}$ is obtained by selecting vertices in V with indegree $\geq k$ and their reachable closure.

This construction establishes a comprehensive unified perspective in which bipartite graphs are treated as a special instance of the hypergraph model. Through the (k, δ) -density decomposition, their hierarchical dense structures are preserved and naturally extended to capture higher-order connectivity patterns. This unification bridges two fundamental graph abstractions under a single, coherent density-based framework, highlighting both the theoretical generality and practical versatility of our approach.

2.7 Problem Statements and Challenges

Problem 1: Dense Subhypergraph Mining (DSM). Given a hypergraph $\mathcal{H} = (V, E)$ and integers k, δ , compute the (k, δ) -dense subhypergraph $D_{k,\delta}$.

Problem 2: Density Subhypergraph Decomposition (DSD). Given a hypergraph $\mathcal{H} = (V, E)$, compute all non-empty (k, δ) -dense subhypergraphs as k and δ varies.

A naive approach to Problem 1 is to iteratively identify reversible hyperpaths and reverse them until an egalitarian orientation is achieved. However, this method may incur a time complexity of $O(2^{m\delta})$ and does not scale to large hypergraphs. To overcome this limitation, we aim to develop more efficient methods for either reversing all reversible hyperpaths or directly computing a relatively fair orientation in which all qualifying vertices have no reversible hyperpaths to any vertex outside the set.

For Problem 2, directly applying the solution to Problem 1 for each (k, δ) pair is computationally inefficient. Therefore, we seek to design a more efficient method for constructing the entire decomposition hierarchy. The key challenges are:

- How to design an algorithm that enables the efficient computation of an egalitarian or relatively fair orientation?
- How to exploit the hierarchical structure of the decomposition to eliminate redundant computation?

Algorithm 1: DSM-PATH($\mathcal{H}, k, \delta$)

Input: a hypergraph $\mathcal{H}$, two non-negative integer k, δ .

Output: (k, δ) -dense subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

```

1 Arbitrarily obtain a  $\delta$ -orientation  $\vec{\mathcal{H}}$  of  $\mathcal{H}$ ;
2 while True do
3   if  $\exists$  a reversible hyperpath  $s \rightsquigarrow t$  then
4      $\quad$  reverse the hyperpath  $s \rightsquigarrow t$ ;
5   else break;
6  $S \leftarrow \{u \in V \mid \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$ ;
7  $D_{k,\delta} \leftarrow S \cup \{v \mid v \text{ can reach a vertex in } S\}$ ;
8 return  $D_{k,\delta}$ ;
```

3 DENSE SUBHYPERGRAPH MINING

This section presents four algorithms for computing the (k, δ) -dense subhypergraph. Naively eliminating all reversible hyperpaths may incur complexity up to $O(2^{m\delta})$. However, by leveraging the properties of hyperpaths, we can start from vertices with in-degree at least k , iteratively search for reversible hyperpaths, and reverse them. Each reverse reduces the in-degree of a high-degree vertex by one, and since the total in-degree is bounded by $O(m\delta)$, and each hyperpath search takes $O(m\delta)$ time, this motivates the DSM-PATH algorithm, which removes one reversible hyperpath per round using BFS. And the overall time complexity is $O(m^2\delta^2)$. To improve efficiency, DSM-FLOW formulates the problem as a maximum flow instance to eliminate all reversible hyperpaths between vertices of in-degree above and below k in a single step. This approach achieves a lower time complexity of $O(m^{1.5}\bar{d}_e^{1.5})$, where $\bar{d}_e$ denotes the average hyperedge size, at the cost of increased memory consumption. Finally, DSM-ALL avoids the memory overhead of network flow formulations while still eliminating all reversible hyperpaths between high- and low-degree vertices. It guarantees local fairness and achieves a time complexity of $O(nm\delta)$.

3.1 The BFS-based algorithm: DSM-PATH

The algorithm performs BFS to identify and reverse a reversible hyperpath. The DSM-PATH algorithm is shown in Algorithm 1. First, it obtains an arbitrarily δ -orientation of $\mathcal{H}$ (Line 1). Then, in the while loop (Lines 2-5), the algorithm finds (Line 3) and reverses (Line 4) all reversible hyperpaths through the *BFS*. Each iteration of the loop invokes one BFS, removing one reversible hyperpath. The loop terminates when no reversible hyperpath can be found (Line 5). Finally, the algorithm obtains the (k, δ) -dense subhypergraph $D_{k,\delta}$ according to the definition (Line 7).

According to Definition 4, the DSM-PATH algorithm correctly outputs the (k, δ) -dense subhypergraph $D_{k,\delta}$.

THEOREM 6 (COMPLEXITY OF ALGORITHM DSM-PATH). *The time and space complexity are $O(m^2\delta^2)$ and $O(n + m)$.*

PROOF. For the arbitrary δ -orientation $\vec{\mathcal{H}}$ obtained by line 1 of Algorithm 1, let $\bar{S} = \{u \in V \mid \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$ as the initial set S . The reversal of a $s \rightsquigarrow t$ hyperpath cannot add any new vertices to the set S , thus every vertex t in the $s \rightsquigarrow t$ hyperpath must be in $\bar{S}$. As each reversal decreases the indegree of a vertex t in $\bar{S}$

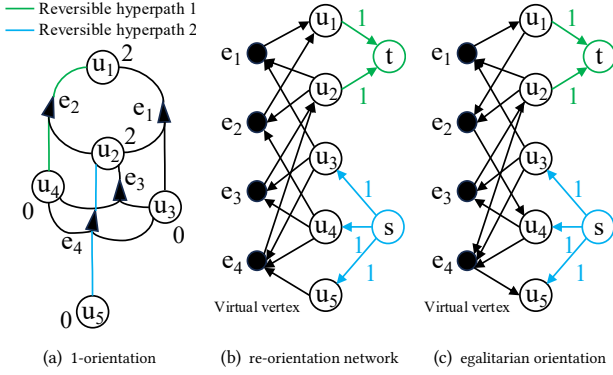


Figure 3: An example of the re-orientation network.

by 1 and the indegree of a vertex is non-negative, the number of reversals is clearly bounded by $\sum_{x \in S} \vec{d}_x \leq m\delta$. Each $s \rightsquigarrow t$ hyperpath can be reversed in $O(m\delta)$ time. Since there are at most $O(m\delta)$ such hyperpaths, the total time complexity is $O(m^2\delta^2)$. The space complexity is linear in the input size, i.e., $O(n + m)$. $\square$

3.2 The flow-based algorithm: DSM-FLOW

To overcome the inefficiency of DSM-PATH, which may reverse up to $O(m\delta)$ hyperpaths, we introduce the DSM-FLOW algorithm to efficiently separate (k, δ) -dense vertices from others. Leveraging the strength of network flow in vertex separation, DSM-FLOW eliminates all relevant reversible $s \rightsquigarrow t$ hyperpaths in one pass via a flow network. The core idea builds on augmenting hyperpath algorithms, adapting max-flow techniques to remove reversible hyperpaths. Inspired by the reorientation network of Bezakova et al. [4], we design a novel reorientation hypergraph network.

DEFINITION 10 (RE-ORIENTATION HYPERGRAPH NETWORK). Given a δ -orientation $\vec{\mathcal{H}} = (V, \vec{E})$ and an integer d , the re-orientation hypergraph network is constructed as a weighted factor graph, in which each hyperedge is treated as a vertex in the network, augmented with an additional source vertex s and sink vertex t . The weight assigned to each arc between two vertices represents the capacity of the arc. Specifically, the re-orientation network with parameter d is defined as $(V \cup V_E \cup \{s, t\}, A, c)$, where

- 1) $\langle u, v_e \rangle \in A, c(u, v_e) = 1$, if $u \in T, v_e \in V_E, \vec{e} = \{T, H\}$;
- 2) $\langle v_e, u \rangle \in A, c(v_e, u) = 1$, if $v_e \in V_E, u \in H, \vec{e} = \{T, H\}$;
- 3) $\langle s, u \rangle \in A, c(s, u) = \vec{d} - \vec{d}_u(\vec{H})$, if $\vec{d}_u(\vec{H}) < d$;
- 4) $\langle u, t \rangle \in A, c(u, t) = \vec{d}_u(\vec{H}) - d$, if $\vec{d}_u(\vec{H}) > d$.

By Definition 10, the re-orientation hypergraph network uses a parameter d to separate vertices based on their indegree. To obtain $D_{k,\delta}$, we set $d = k - 1$. Consequently, the source s connects to vertices with an indegree less than $k - 1$, while the sink t links to vertices with an indegree greater than $k - 1$. Upon completion of the maximum flow algorithm, no augmentation paths remain in the residual network, indicating no reversible hyperpaths from s to t . Therefore, all reversible hyperpaths are reversed and $D_{k,\delta}$ can be obtained.

EXAMPLE 3. Given the 1-orientation in Figure 2(b) and $k = 2$, the corresponding re-orientation hypergraph network is shown in Figure

Algorithm 2: DSM-FLOW($\mathcal{H}, k, \delta$)

Input: a hypergraph $\mathcal{H}$, two non-negative integer k, δ .
Output: (k, δ) -dense subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

- 1 Arbitrarily obtain an δ -orientation $\vec{\mathcal{H}}$ of $\mathcal{H}$;
- 2 $V' \leftarrow V \cup \{s, t\} \cup V_E, d \leftarrow k - 1$;
- 3 **for each** $\langle u, v_e \rangle, u \in T, v_e \in V_E, \vec{e} = \{T, H\} \in \vec{E}$ **do**
- 4 $\mid$ add arc $\langle u, v_e \rangle$ to A and let $c(u, v_e) \leftarrow 1$;
- 5 **for each** $\langle v_e, u \rangle, v_e \in V_E, u \in H, \vec{e} = \{T, H\} \in \vec{E}$ **do**
- 6 $\mid$ add arc $\langle v_e, u \rangle$ to A and let $c(v_e, u) \leftarrow 1$;
- 7 **for each** $u, \vec{d}_u(\vec{H}) < d$ **do**
- 8 $\mid$ add arc $\langle s, u \rangle$ to A and let $c(s, u) \leftarrow d - \vec{d}_u(\vec{H})$;
- 9 **for each** $u, \vec{d}_u(\vec{H}) > d$ **do**
- 10 $\mid$ add arc $\langle u, t \rangle$ to A and let $c(u, t) \leftarrow \vec{d}_u(\vec{H}) - d$;
- 11 Compute the maximum flow value $f_{\max}$ of (V', A, c) ;
- 12 **for each** $(u_x, v_e, u_y), u_x, u_y \in V, v_e \in V_E$ **do**
- 13 **if** $\langle u_x, v_e \rangle \in A, \langle v_e, u_y \rangle \in A$ **are saturated then**
- 14 $\mid$ reverse $\langle u_x, v_e \rangle, \langle v_e, u_y \rangle$; $\mid u_x \in H, u_y \in T$
- 15 Same as lines 6-7 in Algorithm 1;
- 16 **return** $D_{k,\delta}$;

3(b), where the capacity of each arc is 1. For vertices in V , $k - 1$ is the pivot of indegree. Source s is connected to the vertices whose indegree does not reach the pivot, thus it is connected to u_3, u_4, u_5 with a capacity of $k - 1 - \vec{d}_{u_3}/\vec{d}_{u_4}/\vec{d}_{u_5} = 1$. Sink t is connected to the vertices whose indegree exceeds the pivot, thus it is connected to u_1 and u_2 with a capacity of $\vec{d}_{u_1}/\vec{d}_{u_2} - (k - 1) = 1$. After computing maximum flow, the network is shown in Figure 3(c).

We devise the network flow algorithm DSM-FLOW, as shown in Algorithm 2. The algorithm first arbitrarily obtains an orientation (Line 1) and then constructs the re-orientation network (Lines 4-10). After that, it computes the maximum flow on it (Line 11). Then, it reverses all reversible hyperpaths (Lines 12-14). Finally, it obtains $D_{k,\delta}$ and returns it as the (k, δ) -dense subhypergraph $D_{k,\delta}$ (Line 15). Below we prove the correctness and complexity of Algorithm DSM-FLOW.

THEOREM 7 (CORRECTNESS OF ALGORITHM DSM-FLOW). Algorithm DSM-FLOW correctly outputs $D_{k,\delta}$.

PROOF. After computing the maximum flow, we reverse all saturated edges $\langle u_x, u_y \rangle \in \vec{E}$ in the directed hypergraph $\vec{\mathcal{H}}$. The resulting set $S = \{u \in V \mid \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$ includes exactly those vertices reachable from t via unsaturated hyperedges in the residual network. Similarly, $V \setminus S$ includes those connected to s . Since no residual path exists from s to t in network, there is no reversible hyperpath from $V \setminus S$ to S in $\vec{\mathcal{H}}$, satisfying the (k, δ) -dense subhypergraph condition. Thus, the DSM-FLOW is precise. $\square$

THEOREM 8 (COMPLEXITY OF ALGORITHM DSM-FLOW). The time and space complexity are $O(m^{1.5}d_e^{1.5})$ and $O(n + m\vec{d}_e)$.

Algorithm 3: DSM-FLOW+($\mathcal{H}, k, \delta$)**Input:** a hypergraph $\mathcal{H}$, two non-negative integer k, δ .**Output:** (k, δ) -dense subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

```

1  $\vec{E} \leftarrow \emptyset, \vec{\mathcal{H}} \leftarrow (V, \vec{E});$ 
2 for each  $e \in E$  do
3    $V_e \leftarrow$  select  $\delta$  vertices with lowest degree in  $e$ ;
4    $\vec{e} = \{\{e \setminus V_e\}, \{V_e\}\}, \vec{E} \leftarrow \vec{E} \cup \vec{e};$ 
5 Same as lines 2-17 in Algorithm 2;
6 return  $D_{k,\delta};$ 

```

PROOF. As shown in [6], the reorientation network is an AUC-2 network (i.e., unit-capacity edges except for those incident to source/sink). Computing maximum flow on such networks takes $O(|E|^{1.5})$ time and $O(|E|)$ space. Since the network size is scaled by average hyperedge size $\bar{d}_e$, the overall complexity becomes $O(m^{1.5} \bar{d}_e^{1.5})$ in time and $O(n + m \bar{d}_e)$ in space. $\square$

3.3 The improved algorithm: DSM-FLOW+

We propose an enhanced algorithm, DSM-FLOW+, which aims to balance indegree distribution by orienting each hyperedge toward the vertices with the lowest current indegree. This design aligns more directly with the objective of minimizing maximal indegree across the hypergraph. Specifically, DSM-FLOW+ first constructs an orientation $\vec{\mathcal{H}}$, where each hyperedge initially points to the endpoint with the smaller indegree (Algorithm 3 Lines 2–4). DSM-FLOW+ focuses on reducing vertex indegrees, which better aligns with the objective of minimizing the maximum indegree, progressively reducing the maximum indegree of the orientation and thus achieving a more balanced distribution.

THEOREM 9 (COMPLEXITY OF ALGORITHM DSM-FLOW+). *The time and space complexity are $O(m^{1.5} \bar{d}_e^{1.5})$ and $O(n + m \bar{d}_e)$.*

PROOF. The DSM-FLOW+ algorithm modifies DSM-FLOW by introducing a deterministic δ -orientation heuristic. For each hyperedge $e \in E$, selecting the δ vertices with the lowest degree can be performed in $O(|e|)$ time via linear scan or $O(\delta \log |e|)$ via partial heap selection. Since each hyperedge is processed once, the orientation step takes total time $O(m\delta)$. The subsequent steps are identical to DSM-FLOW, which has time complexity $O((m \bar{d}_e)^{1.5})$. Therefore, the total time complexity remains $O(m^{1.5} \bar{d}_e^{1.5})$, and space usage is dominated by the size of the flow network, i.e., $O(n + m \bar{d}_e)$. $\square$

3.4 The improved algorithm: DSM-ALL

Although the construction of a flow network in DSM-FLOW offers significant acceleration, it also incurs substantial memory overhead. Motivated by the core idea of network flow—that is, separating vertices based on hyperpath accessibility—we seek a more lightweight and scalable method for identifying all vertices whose indegree is at least k . To this end, we propose the DSM-ALL algorithm, which efficiently identifies a set of such vertices, ensuring that none of vertex outside the set can reach any vertex in the set via a reversible hyperpath. By treating this set as a cohesive unit, the hypergraph can be regarded as globally fair, since no external vertex can reach

it through a reversible hyperpath, enabling efficient vertex filtering and improving both performance and scalability.

The algorithm DSM-ALL proceeds as Algorithm 4: all vertices with indegree at least k are initially included in the initial set $\bar{S}$ (Line 2). Then, for each vertex outside $\bar{S}$, we search for a reversible hyperpath to vertex $u \in \bar{S}$ and reverse it if found (Lines 10–11). If the reversal increases the indegree of the external vertex to k , it is incorporated into $\bar{S}$ (Line 12). At this stage, the hypergraph becomes relatively fair, as no reversible hyperpaths exist from the outside to the current set. However, in the process of discovering and reversing such cross-boundary hyperpaths, some vertices inside $\bar{S}$ may experience a drop in indegree below k . These vertices must be removed from $\bar{S}$ (Line 5). Prior to removal, we ensure fairness preservation by reversing any remaining reversible hyperpaths between the vertex and its neighbors in $\bar{S}$ (Lines 18–19), thereby eliminating potential violations upon removal. Subsequently, we also check and reverse hyperpaths from the vertex to external vertices (Lines 20–21). If its indegree remains below k after all such operations, the vertex is permanently removed from $\bar{S}$ (Line 22).

THEOREM 10 (CORRECTNESS OF ALGORITHM DSM-ALL). *Algorithm DSM-ALL correctly outputs $D_{k,\delta}$.*

PROOF. It ensures that all vertices in the output $D_{k,\delta}$ satisfy the indegree constraint $\vec{d}_v \geq k$ and eliminates all reversible hyperpaths between qualifying and non-qualifying vertices. The algorithm iteratively prunes any vertex that violates these conditions and terminates when no further reversals are possible. Thus, the output satisfies the definition of a (k, δ) -dense subhypergraph. $\square$

THEOREM 11 (COMPLEXITY OF ALGORITHM DSM-ALL). *The time and space complexity are $O(nm\delta)$ and $O(n + m)$.*

PROOF. The DSM-ALL algorithm computes the (k, δ) -dense subhypergraph by iteratively repairing and pruning vertices based on reversible hyperpaths over a δ -orientation of the input hypergraph $\mathcal{H} = (V, E)$. In the worst case, each vertex may trigger both REACHOUT and OUT operations, each involving hyperpath reversals with cost proportional to the number of incident hyperedges. As a result, the total time complexity is $O(nm\delta)$. The algorithm requires only linear space to store the oriented hypergraph and auxiliary metadata, leading to a space complexity of $O(n + m)$. $\square$

4 DENSITY DECOMPOSITION

We propose two algorithms for computing all non-empty (k, δ) -dense subhypergraphs: a basic iterative algorithm DSD, and an enhanced version DSD+ that leverages divide-and-conquer. DSD incrementally extracts $D_{1,\delta}, D_{2,\delta}, \dots, D_{k_{\max},\delta}$ by repeatedly invoking a subroutine for identifying a single (k, δ) -dense subhypergraph. However, this process incurs substantial redundancy, as many intermediate results are repeatedly recomputed. To address this, DSD+ exploits the hierarchical structure of dense subhypergraphs to reduce overlap: it computes intermediate layers recursively and reuses results across subproblems, thereby improving efficiency without sacrificing completeness.

Algorithm 4: DSM-ALL($\mathcal{H}, k, \delta$)**Input:** a hypergraph $\mathcal{H}$, two non-negative integer k, δ .**Output:** (k, δ) -dense subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

```

1 Arbitrarily obtain a  $\delta$ -orientation  $\vec{\mathcal{H}}$  of  $\mathcal{H}$ ;
2  $\bar{S} \leftarrow \{u \in V \mid \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$ ;
3 for each  $u \in \bar{S}$  do REACHOUT( $u, k$ );
4 while True do
5   if  $\exists u \in \bar{S}$  with  $\vec{d}_u < k$  then OUT( $u, k$ );
6   else break;
7  $D_{k,\delta} \leftarrow \bar{S} \cup \{v \mid v \text{ can reach a vertex in } \bar{S}\}$ ;
8 return  $D_{k,\delta}$ ;
9 Function REACHOUT( $u, k$ ):
10   if  $\exists$  a reversible hyperpath  $s \rightsquigarrow u$  with  $s \in V \setminus \bar{S}$  then
11     reverse the hyperpath  $s \rightsquigarrow u$ ;
12   if  $\vec{d}_s \geq k$  then  $\bar{S} \leftarrow \bar{S} \cup s$ ;
13 Function REACHIN( $u, k$ ):
14   if  $\exists$  a reversible hyperpath  $u \rightsquigarrow s$  with  $s \in V \setminus \bar{S}$  then
15     reverse the hyperpath  $u \rightsquigarrow s$ ;
16 Procedure OUT( $u, k$ ):
17   initindegree  $\leftarrow \vec{d}_u$ ;
18   if  $\exists$  reversible  $s \rightsquigarrow u/u \rightsquigarrow s, s \in \bar{S} \cap N(u)$  then
19     reverse the hyperpath;
20   if initindegree  $> \vec{d}_u$  then REACHIN( $u, k$ );
21   else initindegree  $< \vec{d}_u$  then REACHOUT( $u, k$ );
22   if  $\vec{d}_u < k$  then  $\bar{S} \leftarrow \bar{S} \setminus u$ ;

```

Algorithm 5: DSD($\mathcal{H}$)**Input:** a hypergraph $\mathcal{H} = (V, E)$.**Output:** all non-empty $D_{k,\delta}$ of $\mathcal{H}$

```

1 for  $\delta = 1, 2, \dots$  do
2   Arbitrarily obtain a  $\delta$ -orientation  $\vec{\mathcal{H}}$  of  $\mathcal{H}$ ;
3   for  $k = 1, 2, 3, \dots$  do
4     DSM-ALL( $\vec{\mathcal{H}}, k, \delta$ );
5     if  $D_{k,\delta} = \emptyset$  then break;
6   return  $\mathcal{D} = \{D_{k,\delta}\}$ ;

```

4.1 The basic decomposition algorithm: DSD

Using the (k, δ) -dense subhypergraph mining algorithm DSM-ALL, the density subhypergraph decomposition can be computed layer by layer. Based on this idea, we propose the DSD algorithm as shown in Algorithm 5. Specifically, DSD enumerates all combinations of δ and k through two nested loops (Lines 1-3). After fixing δ and k , the algorithm invokes DSM-ALL($\vec{\mathcal{H}}, k, \delta$) (Line 4).

The correctness of DSD follows directly from DSM-ALL and Theorems 2 and 3, so we omit the proof.

THEOREM 12 (COMPLEXITY OF ALGORITHM DSD). *The time and space complexity are $O(nm\delta \cdot d_{\max}^E \cdot k_{\max})$ and $O(n + m)$.*

PROOF. The DSD algorithm performs a hierarchical (k, δ) -dense decomposition by iteratively applying vertex pruning based on indegree constraints across increasing values of k and δ . For each

δ -orientation (constructed in $O(m\delta)$ time), the algorithm executes a peeling process up to $k_{\max}$ layers, where each layer may invoke up to $O(n)$ calls to the OUT function, with worst-case cost $O(m\delta)$ per call. This results in a total time complexity of $O(nm\delta \cdot d_{\max}^E \cdot k_{\max})$. The space complexity remains linear at $O(n + m)$, as only the oriented hypergraph and auxiliary vertex states are maintained. $\square$

4.2 The improved decomposition algorithm: DSD+

DSD performs layer-by-layer decomposition in a straightforward but computation-intensive manner. It sequentially computes $D_{1,\delta}, D_{2,\delta}, \dots, D_{k_{\max},\delta}$, but due to the nested nature of these subhypergraphs, many computations are repeatedly performed across layers. As k increases, this redundancy accumulates, resulting in considerable inefficiency. To address this, we propose DSD+, a divide-and-conquer variant that reduces redundant computation by reusing intermediate results. Instead of processing all layers sequentially, DSD+ recursively partitions the range. Given a lower bound $D_{k_l,\delta}$ and an upper bound $D_{k_u,\delta}$, it selects a midpoint k_m , computes $D_{k_m,\delta}$ and $D_{k_m+1,\delta}$ using DSM-ALL, and then recursively processes the intervals (k_l, k_m) and $(k_m + 1, k_u)$ within the corresponding subhypergraph differences. This is enabled by the hierarchical property of (k, δ) -dense subhypergraphs, where $D_{k,\delta} \supseteq D_{k+1,\delta}$, ensuring that recursion only explores unexplored regions without redundancy.

The full decomposition for a given δ is obtained by invoking DIVIDE($D_{1,\delta}, D_{k_{\max},\delta}$), where $D_{1,\delta} = V$ and $D_{k_{\max},\delta}$ is the deepest non-empty layer, which can be found via binary search. This process efficiently recovers all non-empty layers $D_{1,\delta}, D_{2,\delta}, \dots$. Then, we develop the DSD+ algorithm (Algorithm 6). First, for any values of $\delta, k_{\max}$ can be determined via binary search (Line 3). Then, the algorithm computes DIVIDE($D_{k_{\max},\delta}, D_{1,\delta}$), and calls DIVIDE to compute dense subhypergraphs (Line 4). When invoking DIVIDE, the algorithm first checks whether the recursion termination condition is reached (Line 7). If not, it proceeds to compute $D_{k_m,\delta}$, and then continues the recursive decomposition (Lines 9–10).

THEOREM 13 (CORRECTNESS OF ALGORITHM DSD+). *Given a hypergraph $\mathcal{H} = (V, E)$, Algorithm DSD+ correctly computes all non-empty (k, δ) -dense subhypergraphs $D_{k,\delta}$, and each such subhypergraph is computed exactly once.*

PROOF. The correctness of DSD+ follows these properties:

(1) **Nestedness.** From Theorem 3, we have $D_{k+1,\delta} \subseteq D_{k,\delta}$ for any k , which means the decomposition forms a hierarchy over k . This guarantees that for any $k_l < k_u$, all intermediate layers $D_{k,\delta}$ with $k_l < k < k_u$ lie between $D_{k_u,\delta}$ and $D_{k_l,\delta}$.

(2) **Recursive completeness:** The divide-and-conquer process guarantees that every k is explored unless the corresponding subhypergraph is empty. The recursion terminates when no finer layer can be distinguished, ensuring that all distinct, non-empty layers are visited exactly once. $\square$

To formally guarantee that Algorithm DSD+ produces the complete (k, δ) -dense decomposition, we establish the following correctness theorem.

THEOREM 14 (COMPLEXITY OF ALGORITHM DSD+). *The time and space complexity are $O(nm\delta \cdot d_{\max}^E \log k_{\max})$ and $O(n + m)$.*

Algorithm 6: DSD+ ($\mathcal{H}$)

Input: a hypergraph $\mathcal{H} = (V, E)$.
Output: all non-empty $D_{k,\delta}$ of $\mathcal{H}$

```

1 for  $\delta = 1, 2, 3, \dots$  do
2   Arbitrarily obtain an  $\delta$ -orientation  $\vec{\mathcal{H}}$  of  $\mathcal{H}$ ,  $D_{1,\delta} \leftarrow V$ ;
3    $k_{max} \leftarrow$  the maximum integral such that  $D_{k_{max},\delta} \neq \emptyset$ ;
4   DIVIDE ( $D_{k_{max},\delta}, D_{1,\delta}$ );
5 return  $\mathcal{D} = \{D_{k,\delta}\}$ ;
6 Function DIVIDE ( $D_{k_u,\delta}, D_{k_l,\delta}$ ):
7   if  $k_u - k_l \leq 1$  or  $D_{k_u,\delta} = D_{k_l,\delta}$  then
8     return;
9    $k_m \leftarrow (k_u + k_l + 1)/2$ , DSM-ALL ( $\vec{\mathcal{H}}, k_m, \delta$ );
10  DIVIDE ( $D_{k_u,\delta}, D_{k_m,\delta}$ ), DIVIDE ( $D_{k_m,\delta}, D_{k_l,\delta}$ );

```

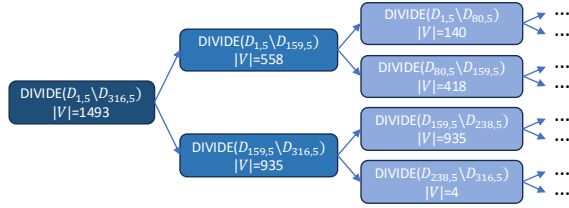


Figure 4: An example of DIVIDE ($D_{1,5}, D_{316,5}$) on HB dataset.

PROOF. The DSD+ algorithm adopts a divide-and-conquer strategy that recursively bisects the density interval and invokes DSM-ALL on intermediate layers. As the recursion depth is $O(\log k_{max})$ and each invocation of DSM-ALL takes $O(nm\delta)$ time, the overall time complexity of DSD+ is $O(nm\delta \cdot d_{max}^E \cdot \log k_{max})$. The space complexity remains linear at $O(n + m)$, as each recursive step operates in-place without duplicating the hypergraph structure. $\square$

Figure 4 illustrates an example of the DIVIDE process applied to the HB dataset, where we recursively decompose the interval $[1, 316]$ with fixed $\delta = 5$. The initial call DIVIDE ($D_{1,5}, D_{316,5}$) covers a vertex set of size 1493. The midpoint $k = 159$ is selected, and the recursion continues on two subintervals: $[1, 159]$ and $[159, 316]$, with corresponding vertex sets of sizes 558 and 935, respectively. Each recursive call further selects a midpoint (e.g., $k = 80$, $k = 238$) and continues the decomposition, splitting the interval until base cases are reached—either when the subinterval length is no more than 1 or when two adjacent layers contain identical vertex sets (e.g., DIVIDE ($D_{238,5}, D_{316,5}$) with only 4 vertices).

This hierarchical tree highlights the core advantage of DSD+: by reusing intermediate results and skipping redundant computations, the algorithm significantly reduces time complexity and accelerates decomposition. As visualized in the figure, lighter-colored blocks indicate smaller vertex sets, intuitively reflecting how the workload shrinks at deeper recursion levels. This divide-and-conquer strategy enables efficient identification of all distinct non-empty (k, δ) -dense subhypergraphs with minimal overhead.

5 PARALLEL AND DYNAMIC ALGORITHMS

We extend DSD+ with parallel and dynamic variants to support large-scale and evolving hypergraphs efficiently.

Algorithm 7: Parallel DSD+

Input: A hypergraph $\mathcal{H} = (V, E)$
Output: All non-empty $D_{k,\delta}$ of $\mathcal{H}$

```

1 Parallel for  $\delta = 1, 2, 3, \dots$  do:
2   Arbitrarily obtain an  $\delta$ -orientation  $\vec{\mathcal{H}}$  of  $\mathcal{H}$ ,  $D_{1,\delta} \leftarrow V$ ;
3    $k_{max} \leftarrow$  the maximum integral such that  $D_{k_{max},\delta} \neq \emptyset$ ;
4   DIVIDE ( $D_{k_{max},\delta}, D_{1,\delta}$ );
5 End Parallel for
6 return  $\mathcal{D} = \{D_{k,\delta}\}$ ;
7 Function DIVIDE ( $D_{k_u,\delta}, D_{k_l,\delta}$ ):
8   if  $k_u - k_l \leq 1$  or  $D_{k_u,\delta} = D_{k_l,\delta}$  then
9     return;
10   $k_m \leftarrow (k_u + k_l + 1)/2$ , DSM-ALL ( $\vec{\mathcal{H}}, k_m, \delta$ );
11  Parallel Tasks:
12    DIVIDE ( $D_{k_u,\delta}, D_{k_m,\delta}$ ), DIVIDE ( $D_{k_m,\delta}, D_{k_l,\delta}$ );
13  Sync Tasks;

```

5.1 Finer-Grained Parallelism

We further enhance DSD+ with a two-level parallel optimization. **(i) δ -level:** computations for different δ values run concurrently, sharing read-only structures while isolating per- δ workspaces. **(ii) Task-level:** inside the recursive DIVIDE, once the midpoint layer $D_{k_m,\delta}$ is obtained, the two subranges $[k_l, k_m]$ and $[k_m, k_u]$ are processed in parallel, each restricted to its own active-vertex mask. Both levels are implemented using OpenMP guided scheduling in a single parallel region to minimize synchronization. Threads use local buffers and merge results once per barrier, ensuring deterministic outputs identical to the serial version. This design achieves an excellent scalability on large datasets (see Exp-4), further accelerating decomposition without affecting correctness or hierarchy.

5.2 Dynamic Maintenance

Real-world hypergraphs evolve as relations emerge or disappear. To adapt efficiently without full recomputation, we design a dynamic maintenance scheme that incrementally updates the egalitarian orientation and IDNs after each edge insertion or deletion.

Edge addition (Algorithm 8) incrementally integrates a new hyperedge e into the current egalitarian orientation $\vec{\mathcal{H}}$. For each δ iteration (Lines 1–3), it extends the orientation by inserting e while maintaining the non-decreasing indegree order of its incident vertices. For every affected vertex $v \in e$ (Line 4), the algorithm checks whether its indegree $\vec{d}_v^\delta$ reaches the limit $\vec{r}_v^\delta$ (Line 5). If so, it searches for a *reversible hyperpath* $s \rightsquigarrow v$ (Line 6) whose reversal restores balance (Line 7); otherwise, it propagates local adjustments to reachable vertices sharing the same IDN with v (Lines 8–11). Finally, the updated indegrees $\vec{r}$ and orientation $\vec{\mathcal{H}}$ are returned (Line 12). Through localized reversals and updates, the algorithm maintains global egalitarianity without full recomputation.

Edge deletion (Algorithm 9) symmetrically removes a hyperedge e from the current egalitarian orientation $\vec{\mathcal{H}}$ while maintaining balanced indegree distribution. For each δ iteration (Lines 1–3), the algorithm deletes e and inspects affected vertices $v \in e$ (Line 4). If

Algorithm 8: Insert($\vec{\mathcal{H}}, \vec{r}, e$)

Input: The egalitarian orientation $\vec{\mathcal{H}}$, the IDNs of all vertices $\vec{r}$, and the hyperedge e to be inserted.

Output: The updated egalitarian orientation $\vec{\mathcal{H}}$ and IDNs $\vec{r}$

```

1 for  $\delta = 1, 2, \dots$  do
2   Suppose  $e = (u_1, u_2, \dots, u_i, \dots), \vec{d}_{u_i} \leq \vec{d}_{u_{i+1}};$ 
3    $\vec{e} = (\{u_{\delta+1}, u_{\delta+2}, \dots\} \{u_1, u_2, \dots, u_\delta\}), \vec{\mathcal{H}} \leftarrow \vec{\mathcal{H}} \cup \vec{e};$ 
4   for each  $v \in \{u_1, u_2, \dots, u_\delta\}$  do
5     if  $\vec{d}_v = \vec{r}_v^\delta + 1$  then
6       if  $\exists$  reversible hyperpath  $s \rightsquigarrow v$ , with  $\vec{d}_s = \vec{r}_v^\delta - 1$  then
7         reverse the hyperpath  $s \rightsquigarrow v$ ;
8       else
9         for each  $w \in \{w | \vec{r}_w^\delta = \vec{r}_v^\delta\}$  and  $w$  can reach  $v$  do
10            $\vec{r}_w^\delta \leftarrow \vec{r}_w^\delta + 1;$ 
11            $\vec{r}_v^\delta \leftarrow \vec{r}_v^\delta + 1;$ 
12 return  $(\vec{\mathcal{H}}, \vec{r});$ 

```

Algorithm 9: Delete($\vec{\mathcal{H}}, \vec{r}, e$)

Input: The egalitarian orientation $\vec{\mathcal{H}}$, the IDNs of all vertices $\vec{r}$, and the hyperedge e to be deleted.

Output: The updated egalitarian orientation $\vec{\mathcal{H}}$ and IDNs $\vec{r}$

```

1 for  $\delta = 1, 2, 3, \dots$  do
2   Suppose  $e$  is oriented as  $\vec{e} = (\{u_{\delta+1}, u_{\delta+2}, \dots\} \{u_1, u_2, \dots, u_\delta\});$ 
3    $\vec{\mathcal{H}} \leftarrow \vec{\mathcal{H}} \setminus \vec{e};$ 
4   for each  $v \in \{u_1, u_2, \dots, u_\delta\}$  do
5     if  $\vec{d}_v = \vec{r}_v^\delta - 2$  then
6        $\exists$  a reversible hyperpath  $v \rightsquigarrow t$ , with  $d_t = \vec{r}_v^\delta;$ 
7       reverse the hyperpath  $v \rightsquigarrow t$ ;
8        $P \leftarrow \{w | \vec{r}_w^\delta = \vec{r}_v^\delta, \text{ and } w \text{ can reach } v \text{ or } t\} \cup v \cup t;$ 
9     else
10       $P \leftarrow \{w | \vec{r}_w^\delta = \vec{r}_v^\delta, \text{ and } w \text{ can reach } v\} \cup v;$ 
11     for each  $w \in P$  do
12       if  $d_w \neq \vec{r}_w^\delta$  and  $w$  can't reach an  $\vec{r}_w^\delta$ -indegree vertex then
13          $\vec{r}_w^\delta \leftarrow \vec{r}_w^\delta - 1;$ 
14 return  $(\vec{\mathcal{H}}, \vec{r});$ 

```

$\vec{d}_v$ decreases by two below its IDN $\vec{r}_v^\delta$ (Line 5), a reversible hyperpath $v \rightsquigarrow t$ must exist such that the indegree difference between v and t equals 2, and this hyperpath is then reversed to restore balance (Lines 6–7). Afterward, all vertices with the same IDN $\vec{r}_v^\delta$ that can reach either v or t are collected into a temporary set P , including v and t (Line 8). Otherwise, P contains vertices with the same IDN that can reach v , including v itself (Lines 9–10). For each $w \in P$, if $\vec{d}_w$ is lower than its IDN and w cannot reach any vertex with indegree $\vec{r}_w^\delta$, its IDN is decreased (Lines 11–13). Finally, the updated orientation and IDNs are returned (Line 14). Through these localized reversals and corrections, the algorithm efficiently preserves global egalitarianity without full recomputation.

Table 1: Data hypergraph $\mathcal{H}$.

Dataset	$ V = n$	$ E = m$	d_{max}^V	d_{max}^E	d_{min}^E	m/n	$\bar{d}_e$
CP	242	12,704	261	5	2	52.5	2.42
SC	282	315	61	31	4	1.12	17.6
SB	294	29,157	3,264	99	2	99.2	9.65
CH	327	7,818	148	5	2	23.9	2.33
HC	1,290	341	45	82	1	0.26	35.2
HB	1,494	60,987	6,167	399	2	40.8	21.9
TC	172,738	233,202	284	85	2	1.35	3.18
AR	2,268,231	4,285,363	28,969	9,350	2	1.88	17.1
SA	15,211,989	1,103,243	356	61,315	2	0.07	23.7

6 EXPERIMENTAL EVALUATION

6.1 Experimental setup

In this section, we evaluate the performance of our algorithms across multiple datasets. All implementations are in C++ (gcc 11.1.0, -O3) and executed on a Linux server with a 2.9 GHz AMD Ryzen 3990X CPU and 256 GB RAM (CentOS 7.9.2, 64-bit).

Datasets. We evaluate our methods on nine benchmark hypergraph datasets widely used in prior decomposition studies. Table 1 summarizes their key statistics. All datasets are publicly available at <https://www.cs.cornell.edu/~arb/data/> and are listed in ascending order of vertex count. Specifically, CP (*contact-primary-school*) and CH (*contact-high-school*) record proximity interactions among students; SC (*senate-committee*) and HC (*house-committees*) capture committee memberships in the US Senate and House of Representatives; SB (*senate-bills*) and HB (*house-bills*) represent bill co-sponsorship networks in the US Congress; TC (*trivago-clicks*) logs hotels co-clicked during online browsing sessions; AR (*amazon-reviews*) groups products reviewed by individual users on Amazon; and SA (*stackoverflow-answers*) aggregates questions answered by the same user on Stack Overflow.

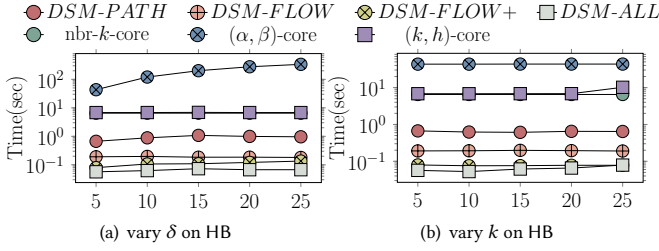
Algorithms We have implemented six algorithms: four (k, δ) -dense subhypergraph algorithms—DSM-PATH (Algorithm 1), DSM-FLOW (Algorithm 2), DSM-FLOW+ (Algorithm 3), and DSM-ALL (Algorithm 4)—as well as two hypergraph density decomposition algorithms, namely DSD (Algorithm 5) and DSD+ (Algorithm 6). It is important to emphasize that our (k, δ) -dense subhypergraph is a novel model specifically designed for hypergraphs, and to the best of our knowledge, no existing algorithms are currently capable of computing (k, δ) -dense subhypergraphs or performing density decomposition in this context. For comparison, we also include several representative hypergraph decomposition algorithms: k -core [27], E-Peel [2] (also referred to as $\text{nbr-}k$ -core), (α, β) -core [14], CoCoreDecomp [28] (i.e., (k, h) -core), densest [21] and HTC-PF (also referred to as hyper k -truss) [33]. To facilitate comparison, we align the parameters across models by mapping k (or α) in these methods to the k in our model, and mapping h , or β to our δ .

6.2 Efficiency Testings

Exp-1: Runtime of subhypergraph mining algorithms. Table 2 and Figure 5 report the runtime of various subhypergraph mining algorithms across multiple datasets and parameter settings. In the table, “OOM” indicates out-of-memory errors, and “UNM” denotes undetermined cases where the runtime exceeds 12 hours. Focusing on the HB dataset with $k = 5$ and $\delta = 5$, all of our algorithms significantly outperform traditional baselines. Specifically,

Table 2: Running time of various methods (second).

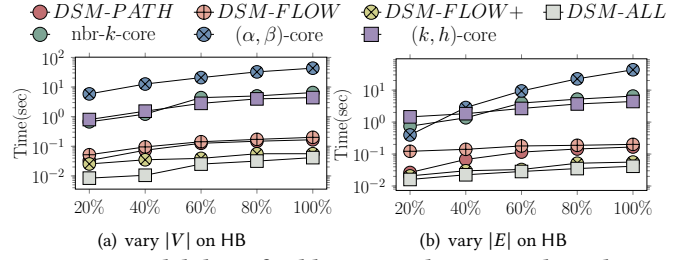
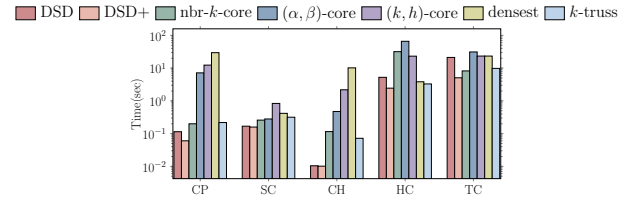
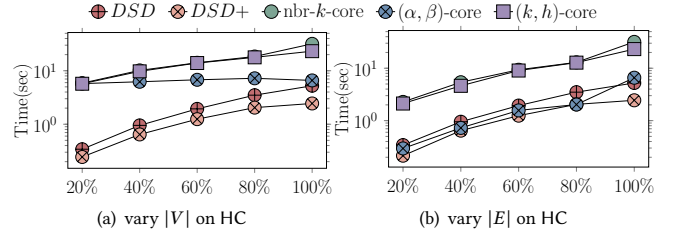
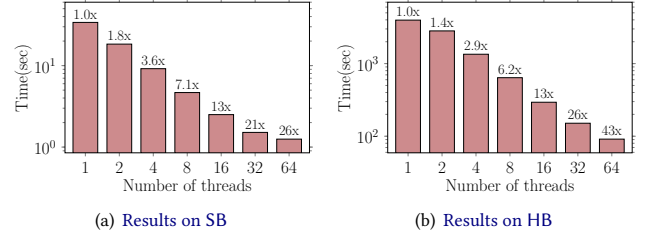
Methods	CP	SB	CH	HC	HB	TC	AR	SA
DSM-PATH	0.007	0.035	0.007	0.002	0.173	273.4	50.07	4.992
DSM-FLOW	0.005	0.032	0.005	0.006	0.151	201.2	OOM	OOM
DSM-FLOW+	0.004	0.015	0.005	0.003	0.055	31.68	OOM	OOM
DSM-ALL	0.002	0.009	0.004	0.002	0.042	0.164	42.17	4.609
nbr-k-core	0.004	0.216	0.005	0.059	6.708	0.594	554.5	2409
(α, β) -core	0.132	14.87	0.031	0.002	43.33	0.236	345.2	UNM
(k, h) -core	0.005	0.377	0.006	0.015	6.828	0.278	170.2	32.34

**Figure 5: Runtime of subhypergraph mining algorithms.**

DSM-ALL achieves the best performance, requiring only 0.042 seconds, while nbr-k-core takes 6.708 seconds—over 160× slower. This trend holds across datasets, where our methods consistently exhibit high efficiency, typically completing within seconds and delivering substantial speedups over classical models. Figure 5 further analyzes runtime trends as δ varies (fixed $k = 5$) and as k varies (fixed $\delta = 5$). In Figure 5(a), as δ increases, classical models experience exponential growth in runtime due to denser substructure enumeration and more complex pruning. In contrast, DSM-ALL maintains nearly constant runtime, demonstrating excellent scalability with respect to δ . Similarly, Figure 5(b) shows that increasing k also leads to significant slowdowns in traditional models, while DSM-ALL remains the most stable, benefitting from its layer-containment property and efficient pruning strategy. Compared to DSM-PATH and DSM-FLOW, which rely on fine-grained hyperpath reversals or network flow computations, DSM-ALL achieves superior scalability and robustness across both dimensions.

Exp-2: Scalability of subhypergraph mining algorithms. Figure 6 evaluates the scalability of subhypergraph mining algorithms on the HB dataset by varying $|V|$ and $|E|$. In both settings, DSM-ALL demonstrates the best scalability, consistently maintaining sub-second runtimes and showing only modest growth as the hypergraph size increases. This indicates its strong resilience to both vertex expansion and edge densification. Other proposed methods, including DSM-FLOW+ and DSM-PATH, also remain efficient and scale reasonably well, though they incur slightly higher costs due to more fine-grained operations such as hyperpath reversals or flow computations. In contrast, classical models such as (α, β) -core and (k, h) -core exhibit steep runtime increases with growing input hypergraph size, making them impractical for large-scale hypergraphs. These results clearly highlight the superior efficiency and robustness of our proposed methods—particularly DSM-ALL—in handling high-volume, high-complexity hypergraph.

Exp-3: Runtime of decomposition algorithms. Figure 7 compares the runtime of seven decomposition algorithms across multiple hypergraph datasets. Our methods, DSD and DSD+, consistently achieve lower runtimes, showing strong efficiency and scalability.

**Figure 6: Scalability of subhypergraph mining algorithms.****Figure 7: Runtime of decomposition algorithms.****Figure 8: Scalability of decomposition algorithms.****Figure 9: Parallel performance.**

In particular, DSD+ attains notable speedups by reusing intermediate results via divide-and-conquer. On TC, nbr-k-core performs slightly faster due to its local peeling design, while the densest model runs faster than DSD on HC as it extracts only a single maximum-density subhypergraph. The k -truss method is also competitive on HC and TC thanks to parallelization (64 threads). In contrast, (α, β) -core and (k, h) -core incur heavy overhead from global computations and complex auxiliary structures.

Exp-4: Scalability and parallelizability. Figure 8 evaluates the scalability of decomposition algorithms on the HC dataset by varying $|V|$ and $|E|$. As $|V|$ increases, DSD and DSD+ show the most stable growth, highlighting their efficiency on large graphs. In contrast, baselines such as (k, h) -core and nbr-k-core slow down sharply with graph size. When varying $|E|$, DSD+ consistently

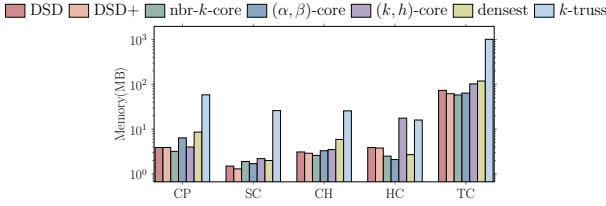


Figure 10: Memory overheads of decomposition algorithms.

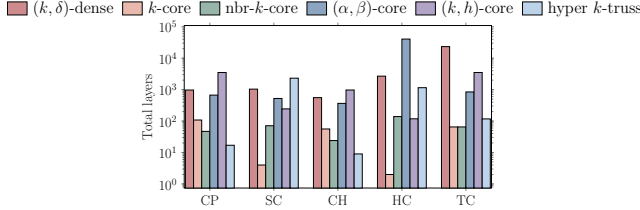


Figure 11: Comparisons of the total hierarchy layers.

achieves the lowest runtime across densities, showing strong robustness and effective redundancy reduction. Overall, these results confirm the excellent scalability and efficiency of our framework. We also evaluate the parallel performance of DSD+ by varying the number of threads from 1 to 64 (Figure 9). As the number of threads increases, the runtime consistently decreases, achieving up to $26\times$ and $43\times$ speedups on the SB and HB datasets, respectively. Similar results can also be observed on the other datasets. These improvements demonstrate the effectiveness of our two-level parallelization, where δ -level and task-level computations are executed concurrently. Overall, the results confirm the excellent parallel scalability of the proposed algorithms.

Exp-5: Memory overheads of decomposition algorithms. Figure 10 compares memory overheads of different decomposition algorithms across real-world hypergraphs. Our methods, DSD and DSD+, maintain stable and low memory usage, reflecting the lightweight design that avoids storing deep hierarchies or redundant metadata. In contrast, core-based methods like (k, h) -core and (α, β) -core consume more and less stable memory, especially on HC and TC. The *densest* baseline adds modest overhead for global density tracking, while *k-truss* shows the highest memory cost due to motif counting and multi-threaded edge-support maintenance.

6.3 Effectiveness Testings

Exp-6: Comparisons of the total hierarchy layers. Figure 11 compares the total number of hierarchy layers generated by different decomposition models across five datasets. Our (k, δ) -dense decomposition generally yields deeper hierarchies—achieving the largest layer count on TC—and is competitive on SC. In contrast, single-parameter methods—such as *k-core*, *nbr-k-core*, and *hyper k-truss*—tend to produce fewer layers, reflecting coarser granularity and limited resolution. Notably, the maximum layer count on SC is attained by *hyper k-truss*, largely because this dataset contains uniformly large hyperedges. Dual-parameter models like (k, h) -core and (α, β) -core also yield relatively large layer counts. However, the layers produced by (k, h) -core and (α, β) -core often

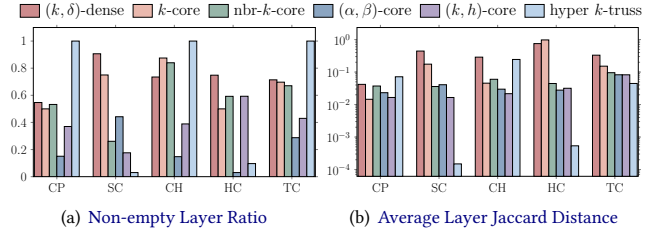


Figure 12: Comparisons of the subhypergraph models.

Table 3: Maximum density of subhypergraph models.

Density Metrics	Methods	CP	SC	CH	HC	HB	TC
$ E / V $	<i>k-core</i>	54.47	1.128	25.58	0.750	38.20	17.78
	<i>nbr-k-core</i>	53.67	1.128	25.40	0.260	37.90	2.804
	<i>hyper k-truss</i>	50.40	1.124	21.62	0.257	OOM	1.510
	<i>densest</i>	54.48	1.128	25.60	1	38.20	UNM
	(k, δ) -dense	54.48	1.128	25.60	0.692	38.20	18.35
volume-density	<i>k-core</i>	70.51	105.5	36.73	195.6	667.2	30.77
	<i>nbr-k-core</i>	71.01	108.2	36.89	216.4	676.8	88.63
	<i>k-truss</i>	67.31	105.4	33.20	197.0	OOM	44.98
	<i>densest</i>	71.01	108.3	36.89	216.4	UNM	UNM
	(k, δ) -dense	70.25	105.5	36.74	195.6	666.9	32.31

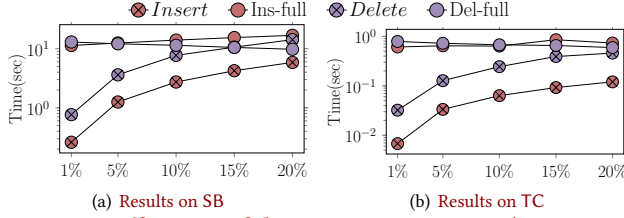
suffer from redundancy, offering limited additional insight. Overall, these results demonstrate that the (k, δ) -dense model offers a balanced tradeoff—providing fine-grained, stable, and expressive hierarchical structure without over-fragmentation or instability, making it well-suited for analyzing complex hypergraphs.

Exp-7: Layer quality comparison. Figure 12 compares decomposition models using two quality metrics: (a) the non-empty layer ratio, reflecting continuity, and (b) the average Jaccard distance between adjacent layers, indicating distinctness. The (k, δ) -dense decomposition consistently yields a high proportion of non-empty layers, indicating that its deeper hierarchies contain meaningful vertex groups. Although (α, β) -core and (k, h) -core produce many layers, their ratios are lower due to fragmented boundaries. On CH, *k-core* and *nbr-k-core* perform slightly better owing to dense vertex connections. For Jaccard distance (Figure 12(b)), (k, δ) -dense almost ranks highest, reflecting clear and non-redundant transitions. *Hyper k-truss* attains the high non-empty ratio and continuity on CP, CH, and TC, largely because these datasets are relatively smaller, allowing motif-based peeling to more precisely capture cohesive substructures. Overall, (k, δ) -dense decomposition produces continuous, distinctive, and robust hierarchies across hypergraphs.

Exp-8: Maximum density of subhypergraph models. We evaluate each subhypergraph model by its ability to capture dense structures under two complementary metrics: edge-vertex ratio ($|E|/|V|$) and volume density [2], which measures the average neighborhood size within the induced subhypergraph. Table 3 reports the maximum density achieved by each model across datasets. For decomposition-based methods (*k-core*, *nbr-k-core*, *hyper k-truss*, and (k, δ) -dense), we report the densest layer, while the *densest* baseline [21] is adapted to the volume-density metric for fair comparison. As shown in Table 3, our (k, δ) -dense model attains the highest edge-vertex density on most datasets. The exception in HC is due to the presence of a single-node hyperedge, which the *densest* baseline trivially selects as the optimal substructure. Moreover,

Table 4: Effect of δ on $k_{\max}$, Sat, and Cont across datasets.

Datasets	Metrics	1	$\lfloor \sqrt{d_e} \rfloor$	d_{50}	$\lfloor \bar{d}_e \rfloor$	d_{75}	d_{95}	$d_{\max}^E$
SB	$k_{\max}$	84	279	734	1138	1374	2695	3264
	Sat	1	0.689	0.442	0.308	0.249	0.048	0
	Cont	0.996	0.988	0.993	0.995	0.996	0.998	0.998
HB	$k_{\max}$	40	243	804	1790	2126	4636	6179
	Sat	1	0.712	0.477	0.291	0.243	0.049	0
	Cont	0.981	0.979	0.992	0.996	0.997	0.998	0.999
TC	$k_{\max}$	19	19	101	179	179	262	284
	Sat	1	1	0.474	0.248	0.248	0.040	0
	Cont	0.662	0.662	0.897	0.941	0.941	0.958	0.961

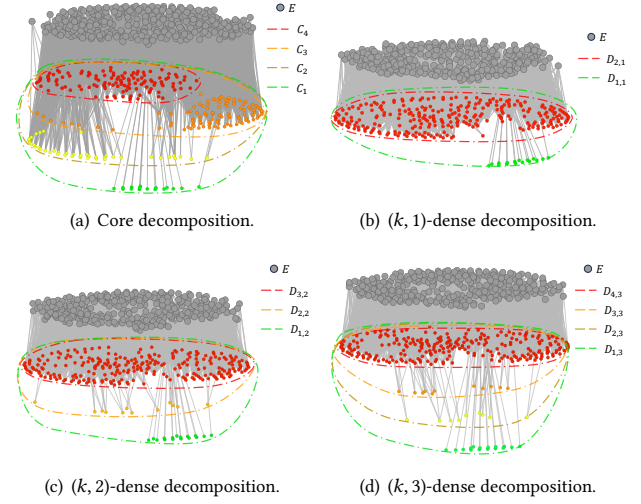
**Figure 13: Efficiency of dynamic maintenance (runtime vs. hyperedge update ratios).**

the (k, δ) -dense model also attains strong volume density performance, while nbr- k -core appears competitive in this metric due to its neighborhood-based formulation.

Exp-9: Ablation on δ and practical guidance. To investigate how the parameter δ affects decomposition, we evaluate three metrics: the number of layers ($k_{\max}$), hyperedge saturation ratio (Sat), and inter-layer continuity (Cont). Sat measures the fraction of truncated hyperedges, while Cont quantifies smoothness between consecutive layers via average Jaccard similarity. As shown in Table 4, δ governs the trade-off between coverage and compactness. When $\delta = 1$, nearly all edges are truncated (Sat ≈ 1), yielding coarse structures. As δ increases, Sat decreases and Cont approaches 1.0, indicating smoother, more interpretable hierarchies. Across datasets, moderate δ values around $\lfloor \bar{d}_e \rfloor$ or d_{75} achieve the best balance (Sat ≈ 0.2 – 0.4 , Cont > 0.9), suggesting a practical rule: set δ such that the saturation ratio falls within 20–40% for stable, well-layered decompositions without excessive computation.

Exp-10: Dynamic maintenance. We simulate dynamic environments by randomly generating update batches containing 1–20% of the original hyperedges (x-axis), including both insertions and deletions. Figure 13 compares the runtime of incremental maintenance and full recomputation on the SB and TC datasets. To avoid the high cost of evaluating all δ values, we fix $\delta = \lfloor \bar{d}_e \rfloor$, a representative setting that yields strong decomposition quality (Exp-8). The incremental method consistently outperforms full recomputation for insertions across all update ratios, exhibiting near-linear scalability. For deletions, it remains faster when updates are small ($\leq 10\%$), but the advantage narrows as deletions increase, with performance converging at a 20% ratio. Overall, these results confirm that our approach efficiently handles frequent, small-scale updates in evolving hypergraphs, while full recomputation is only competitive under large update batches (e.g., beyond 20% of hyperedges are updated).

Exp-11: Visualization of dense subhypergraph models. We conduct a case study on the SC dataset, where the vertices represent

**Figure 14: Core and density decomposition on SC.**

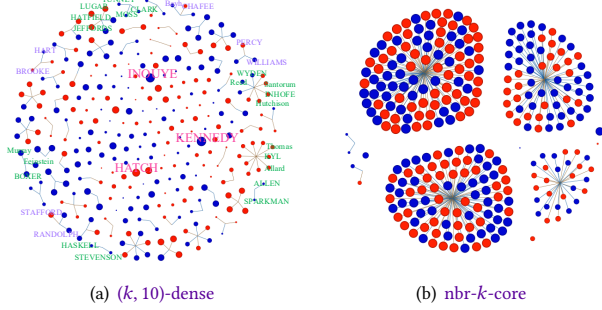
members of the U.S. Senate, and each hyperedge corresponds to a committee membership. We compute multiple dense subhypergraph decompositions, including the traditional k -core decomposition (C_k) and our proposed (k, δ) -dense decomposition ($D_{k,\delta}$). The results are visualized in Figure 14, where Figure 14(a) shows the k -core decomposition, and Figures 14(b)–14(d) present the (k, δ) -dense decompositions. The parameters are selected up to their maximum values, beyond which the corresponding C_k or $D_{k,\delta}$ subhypergraphs become empty. As illustrated, $D_{2,1}$ identifies a tightly connected core structure with significantly higher density and lower conductance than the corresponding C_4 subhypergraph, indicating stronger internal cohesion and weaker external ties. Moreover, we observe that as δ increases, the (k, δ) -dense model reveals progressively more fine-grained and compact substructures. For example, $D_{3,2}$ and $D_{4,3}$ focus even more sharply on high-cohesion regions, producing cleaner boundaries between layers and better filtering out peripheral vertices. In contrast, the k -core decomposition tends to yield coarser partitions with more overlap between layers, missing some of the nuanced structural differences captured by $D_{k,\delta}$. This visualization confirms that (k, δ) -dense decomposition provides more expressive representations of dense subhypergraphs.

6.4 Case studies

Case Study A: Detecting Fraud in Multi-party Financial Transactions. We evaluate our method on the AMLSim benchmark [1, 24], a public simulator that generates anti-money-laundering (AML) transaction graphs. Each alert represents a set of related transactions corresponding to a known fraud pattern, and all participating accounts are labeled as fraudulent. We model each alert as a hyperedge connecting all involved accounts, with additional background edges from normal transactions. This enables quantitative evaluation against ground-truth labels. We apply the $(k, 5)$ -dense decomposition ($\delta = 5 = \lfloor \bar{d}_e \rfloor$, validated in Exp-9) along with k -core and nbr- k -core baselines. Accounts from higher layers (larger k) are ranked and selected as top candidates under limited audit

Table 5: Top-50 Fraud Screening on AMLSim.

Methods	precision	recall	F1	ROC_AUC	PR_AUC
$(k, 5)$ -dense	0.860	0.026	0.050	0.519	0.191
k -core	0.800	0.020	0.042	0.519	0.189
nbr- k -core	0.516	0.020	0.030	0.563	0.170

**Figure 15: Co-sponsorship Community Visualization.**

budgets. Detection quality is assessed using standard metrics: precision, recall, F1, ROC-AUC, and PR-AUC [1]. ROC-AUC measures ranking quality over all thresholds, while PR-AUC better reflects performance under severe class imbalance. As shown in Table 5, the $(k, 5)$ -dense model attains the best precision, F1, and PR-AUC, outperforming k -core and nbr- k -core. Notably, precision improves by 7.5% over k -core under the same Top-50 audit budget, demonstrating that dense layers more effectively concentrate fraudulent accounts for real-world AML screening.

Case Study B: Legislative Co-sponsorship Community Discovery. We evaluate our method on the U.S. Senate Bill dataset, where each bill forms a hyperedge linking all sponsors and legislators are labeled by party affiliation. Legislators (vertices) are labeled by party affiliation (red: Republicans, blue: Democrats). This dataset reflects real political collaborations and serves to assess both interpretability and predictive capability. Figure 15 compares our (k, δ) -dense decomposition ($\delta = 10 = \lfloor \bar{d}_e \rfloor$) with k -core and nbr- k -core. The (k, δ) -dense layers reveal fine-grained communities that distinguish parties and highlight influential bipartisan legislators (*Hatch, Kennedy, Inouye*), within-party groups (*Murray, Feinstein, Boxer*), and cross-party collaborations. In contrast, baseline methods collapse these into coarse layers, obscuring structure. To test predictive power, we train on the first 80% of bills and predict co-sponsorships in the remaining 20%. Evaluation uses two standard link-prediction metrics—AUC (ROC area) and AP [42, 46]. The $(k, 9)$ -dense model attains AUC = 0.865 and AP = 0.873, outperforming k -core (0.820/0.863) and nbr- k -core (0.827/0.878), improving AUC by about 4 ~ 5 points. These results show that our model reveals collaboration hierarchies consistent with real political ties and improves the prediction of future co-sponsorships.

6.5 Extension to Bipartite Graphs.

To demonstrate the extensibility of our framework, we further evaluate it on two representative bipartite datasets, *Answers* and *Bookmark* (<https://files.grouplens.org/datasets/>). Each dataset is

Table 6: Predicting Future Co-sponsorship.

Method	AUC	AP
$(k, 9)$ -dense	0.865	0.873
k -core	0.820	0.863
nbr- k -core	0.827	0.878

Table 7: Results of bipartite extensions.

Dataset	Method	L	L_{eff}/L	$E@L^*$	Motif Density
Answers	<i>Bi-core</i>	20	0.650	1.000	0.334
	<i>k-tip</i>	4	0.250	0.065	0.002
	(k, δ) -dense	100	1.000	1.000	2.043
Bookmark	<i>Bi-core</i>	29	0.931	1.000	0.123
	<i>k-tip</i>	3486	0.507	0.787	0.001
	(k, δ) -dense	2712	0.928	1.000	0.180

modeled as a bipartite graph $G = (U, V, E)$, where U and V are disjoint vertex sets (e.g., movies/tags and users), and $(u, v) \in E$ denotes an interaction. We compare our (k, δ) -dense decomposition against two baselines: (1) *Bi-core*[14], an (α, β) -core extension for bipartite graphs, and (2) *k-tip*[36], which identifies dense butterfly-based substructures. We assess four structural and motif-level metrics: the total number of layers (L), the ratio of effective layers (L_{eff}/L), where a layer is counted as effective only if it contains sufficiently many nodes on both sides, thus reflecting the cleanliness of the hierarchy, edge coverage of top layers ($E@L^*$), and inner-layer motif density ($B_{dens}^{inner, \sigma}$) capturing local butterfly cohesion. As shown in Table 7, the (k, δ) -dense model attains the highest structural compactness ($E@L^* = 1.000$) and motif cohesion (*Motif Density* ≈ 2.0 on *Answers*), while maintaining interpretable hierarchies ($L_{eff}/L \approx 1.0$). Compared with *Bi-core* and *k-tip*, our method forms more meaningful layers—fewer noisy micro-layers in sparse graphs (e.g., *Bookmark*) and finer granularity in dense ones (e.g., *Movie-Lens*). Overall, these results confirm that (k, δ) -dense decomposition generalizes naturally to bipartite structures, preserving both hierarchical interpretability and motif-level fidelity.

7 RELATED WORK

7.1 Graph Decomposition

Subgraph decomposition has been extensively studied, with density-based methods proposed for various types of graphs, including general graphs [45] and bipartite graphs [44]. In addition to density decomposition, numerous approaches have been developed to partition graphs into hierarchies of nested cohesive subgraphs. Representative methods include the k -core decomposition [3, 26], k -truss decomposition [22, 41], nucleus decomposition [37, 38], k -edge-connected decomposition [9, 10], locally densest subgraph (LDS) decomposition [34, 40], distance-generalized core decomposition [7, 13], and colorful h -star k -core decomposition [15, 16].

7.2 Hypergraph Decomposition

Hypergraph Core Decomposition. Numerous models have been proposed for hypergraph core decomposition. Among them, the two most fundamental are the degree-based k -core model [5, 35] and the nbr- k -core model [2, 43]. Building on these foundational models,

several multi-parameter variants have been developed to capture more fine-grained structural characteristics. Some of these variants extend neighborhood-based constraints—such as the (k, h) -core model [28], which incorporates vertex degree, and the (k, g) -core model [25], which considers vertex co-occurrence. Others enhance degree-based formulations by introducing additional factors, including the (k, q) -core [29], which takes hyperedge size into account, and the (k, t) -core [8], which reflects the proportion of vertices involved in each hyperedge.

Other Hypergraph Decomposition. Various aspects of hypergraph decomposition have also been explored. For example, an F -decomposition of a hypergraph G refers to a collection of subhypergraphs of G , each isomorphic to a fixed hypergraph F , such that every edge of G is included in exactly one copy of F [17]. The k -hinge tree decomposition, introduced in [23], has since been applied to the analysis of constraint satisfaction problems. In addition, several generalizations of hypergraph acyclicity have been proposed by defining different forms of hypergraph decompositions, each characterized by a specific notion of width [11, 18]. Intuitively, the width quantifies how far a hypergraph deviates from being acyclic, with a width of 1 corresponding to fully acyclic hypergraphs. Among these, the most prominent decomposition frameworks are hypertree decompositions [19], generalized hypertree decompositions [19], and fractional hypertree decompositions [20].

8 CONCLUSION

In this paper, we conducted a comprehensive study of density decomposition on hypergraphs. Existing approaches often fail to accurately capture the densest subhypergraphs or to flexibly adjust the depth of hierarchical structures. To address these challenges, we proposed the (k, δ) -dense subhypergraph model, which enhances density-aware decomposition through hypergraph redirection and enables adaptive, fine-grained hierarchical structuring, thereby unifying degree-based cores and densest-subgraph formulations under a single cohesive framework. We further developed four subhypergraph mining strategies that leverage hyperpath structures. Furthermore, we design two decomposition algorithms for identifying (k, δ) -dense subhypergraphs, based on network flow optimization. Extensive experiments on nine real-world datasets verified both the effectiveness and scalability of the proposed framework.

REFERENCES

- [1] Erik R. Altman, Jovan Blanus, Luc von Niederhäusern, Beni Egressy, Andreea Anghel, and Kubilay Atasu. 2023. Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- [2] Naheed Anjum Arafat, Arijit Khan, Arpit Kumar Rai, and Bishwamittra Ghosh. 2023. Neighborhood-based Hypergraph Core Decomposition. *VLDB* 16, 9 (2023), 2061–2074.
- [3] Vladimir Batagelj and Matjaz Zaversnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. *CoRR* cs.DS/0310049 (2003).
- [4] Ivona Bezáková. 2000. Compact representations of graphs and adjacency testing. (2000).
- [5] Ginestra Bianconi and Sergey N Dorogovtsev. 2024. Nature of hypergraph k -core percolation problems. *Physical Review E* 109, 1 (2024), 014307.
- [6] Markus Blumenstock. 2016. Fast Algorithms for Pseudoarboricity. In *Proceedings of the Eighteenth Workshop on Algorithm Engineering and Experiments, ALENEX 2016, Arlington, Virginia, USA, January 10, 2016*. SIAM, 113–126.
- [7] Francesco Bonchi, Arijit Khan, and Lorenzo Severini. 2019. Distance-generalized Core Decomposition. In *SIGMOD*. 1006–1023.
- [8] Fanchen Bu, Geon Lee, and Kijung Shin. 2023. Hypercore decomposition for non-fragile hyperedges: concepts, algorithms, observations, and applications. *Data Min. Knowl. Discov.* 37, 6 (2023), 2389–2437.
- [9] Lijun Chang and Zhiyi Wang. 2024. A near-optimal approach to edge connectivity-based hierarchical graph decomposition. *VLDB J.* 33, 1 (2024), 49–71.
- [10] Lijun Chang, Jeffrey Xu Yu, Lu Qin, Xuemin Lin, Chengfei Liu, and Weifa Liang. 2013. Efficiently computing k -edge connected components via graph decomposition. In *SIGMOD*. ACM, 205–216.
- [11] David A. Cohen, Peter Jeavons, and Marc Gyssens. 2008. A unified theory of structural tractability for constraint satisfaction problems. *J. Comput. Syst. Sci.* 74, 5 (2008), 721–743.
- [12] Martina Contisciani, Federico Battiston, and Caterina De Bacco. 2022. Principled inference of hyperedges and overlapping communities in hypergraphs. *CoRR* abs/2204.05646 (2022).
- [13] Qiangqiang Dai, Rong-Hua Li, Lu Qin, Guoren Wang, Weihua Yang, Zhiwei Zhang, and Ye Yuan. 2021. Scaling Up Distance-generalized Core Decomposition. In *CIKM*. 312–321.
- [14] Danhao Ding, Hui Li, Zhipeng Huang, and Nikos Mamoulis. 2017. Efficient Fault-Tolerant Group Recommendation Using α -beta-core. In *CIKM (CIKM '17)*. Association for Computing Machinery, 2047–2050.
- [15] Sen Gao, Rong-Hua Li, Hongchao Qin, Hongzhi Chen, Ye Yuan, and Guoren Wang. 2022. Colorful h -star Core Decomposition. In *ICDE*. 2588–2601.
- [16] Sen Gao, Hongchao Qin, Rong-Hua Li, and Bingsheng He. 2023. Parallel Colorful h -star Core Maintenance in Dynamic Graphs. *VLDB* 16, 10 (2023), 2538–2550.
- [17] Stefan Glock, Daniela Kühn, and Deryk Osthus. 2021. Extremal aspects of graph and hypergraph decomposition problems. In *BCC*. 235–266.
- [18] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2000. A comparison of structural CSP decomposition methods. *Artif. Intell.* 124, 2 (2000), 243–282.
- [19] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2002. Hypertree Decompositions and Tractable Queries. *J. Comput. Syst. Sci.* 64, 3 (2002), 579–627.
- [20] Martin Grohe and Daniel Marx. 2017. Constraint Solving via Fractional Edge Covers. *CoRR* abs/1711.04506 (2017).
- [21] Shuguang Hu, Xiaowei Wu, and T.-H. Hubert Chan. 2017. Maintaining Densest Subsets Efficiently in Evolving Hypergraphs. In *CIKM 2017*. 929–938.
- [22] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k -truss community in large and dynamic graphs. In *SIGMOD*. ACM, 1311–1322.
- [23] Peter Jeavons, David Cohen, and Marc Gyssens. 1994. A structural decomposition for hypergraphs. *Contemp. Math.* 178 (1994), 161–161.
- [24] Rasmus Ingemann Tuffveson Jensen, Joras Ferwerda, Kristian Sand Jørgensen, Erik Rathje Jensen, Martin Borg, Morten Persson Krogh, Jonas Brunholm Jensen, and Alexandros Iosifidis. 2023. A synthetic data set to benchmark anti-money laundering methods. *Scientific data* 10, 1 (2023), 661.
- [25] Dahee Kim, Junghoon Kim, Sungsu Lim, and Hyun Ji Jeong. 2023. Exploring Cohesive Subgraphs in Hypergraphs: The (k, g) -core Approach. In *CIKM*. 4013–4017.
- [26] Rong-Hua Li, Jeffrey Xu Yu, and Rui Mao. 2014. Efficient Core Maintenance in Large Dynamic Graphs. *IEEE Trans. Knowl. Data Eng.* 26, 10 (2014), 2453–2465.
- [27] Qi Luo, Dongxiao Yu, Zhipeng Cai, Xuemin Lin, and Xiuzhen Cheng. 2021. Hypercore Maintenance in Dynamic Hypergraphs. In *ICDE*. 2051–2056.
- [28] Qi Luo, Dongxiao Yu, Yu Liu, Yanwei Zheng, Xiuzhen Cheng, and Xuemin Lin. 2023. Finer-Grained Engagement in Hypergraphs. In *ICDE*. 423–435.
- [29] Qi Luo, Wenjie Zhang, Zhengyi Yang, Dong Wen, Xiaoyang Wang, Dongxiao Yu, and Xuemin Lin. 2024. Hierarchical Structure Construction on Hypergraphs. In *CIKM*. 1597–1606.
- [30] Marco Mancastroppa, Iacopo Iacopini, Giovanni Petri, and Alain Barrat. 2023. Hyper-cores promote localization and efficient seeding in higher-order processes. *CoRR* abs/2301.04235 (2023).
- [31] Cheng Qian, Dandan Zhao, Ming Zhong, Hao Peng, and Wei Wang. 2024. Cascading failures on interdependent hypergraph. *Communications in Nonlinear Science and Numerical Simulation* 138 (2024), 108237.
- [32] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, and Yongheng Dai. 2023. Explainable Hyperlink Prediction: A Hypergraph Edit Distance-Based Approach. In *ICDE*. IEEE, 245–257.
- [33] Hongchao Qin, Guang Zeng, Ronghua Li, Longlong Lin, Ye Yuan, and Guoren Wang. 2025. Truss Decomposition in Hypergraphs. *Proc. VLDB Endow.* 18, 7 (2025), 2185–2197.
- [34] Lu Qin, Rong-Hua Li, Lijun Chang, and Chengqi Zhang. 2015. Locally Densest Subgraph Discovery. In *SIGKDD*. 965–974.
- [35] Emad Y. Ramadan, Arijit Tarafdar, and Alex Pothén. 2004. A Hypergraph Model for the Yeast Protein Complex Network. In *IPDPS*.
- [36] Ahmet Erdem Sariyüce and Ali Pinar. 2018. Peeling Bipartite Networks for Dense Subgraph Discovery. In *WSDM*. ACM, 504–512.
- [37] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Ümit V. Çatalyürek. 2015. Finding the Hierarchy of Dense Subgraphs using Nucleus Decompositions. In *WWW*. 927–937.
- [38] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and Practically Efficient Parallel Nucleus Decomposition. *VLDB* 15, 3 (2021), 583–596.
- [39] Hanlin Sun and Ginestra Bianconi. 2021. Higher-order percolation processes on multiplex hypergraphs. *Physical Review E* 104, 3 (2021), 034306.
- [40] Nikolaj Tatti. 2019. Density-Friendly Graph Decomposition. *ACM Trans. Knowl. Discov. Data* 13, 5 (2019), 54:1–54:29.
- [41] Jia Wang and James Cheng. 2012. Truss Decomposition in Massive Networks. *VLDB* 5, 9 (2012), 812–823.
- [42] Simon Zhang, Cheng Xin, and Tamal K. Dey. 2024. Expressive Higher-Order Link Prediction through Hypergraph Symmetry Breaking. *Trans. Mach. Learn. Res.* 2024 (2024).
- [43] Wenqian Zhang, Zhengyi Yang, Dong Wen, Wentao Li, Wenjie Zhang, and Xuemin Lin. 2025. Accelerating Core Decomposition in Billion-Scale Hypergraphs. *Proc. ACM Manag. Data* 3, 1 (2025), 6:1–6:27.
- [44] Yalong Zhang, Rong-Hua Li, Qi Zhang, Hongchao Qin, Lu Qin, and Guoren Wang. 2025. Density Decomposition of Bipartite Graphs. *Proc. ACM Manag. Data* 3, 1 (2025), 30:1–30:25.
- [45] Yalong Zhang, Ronghua Li, Qi Zhang, Hongchao Qin, and Guoren Wang. 2024. Efficient Algorithms for Density Decomposition on Large Static and Dynamic Graphs. *Proc. VLDB Endow.* 17, 11 (2024), 2933–2945.
- [46] Zijuan Zhao, Kai Yang, and Jinli Guo. 2024. Heterogeneous hypergraph representation learning for link prediction. *The European Physical Journal B* 97, 10 (2024), 153.